

# Intermediate Machine Learning: Assignment 3

## Deadline

Assignment 3 is due Thursday, October 30 by 11:59pm. Late work will not be accepted as per the course policies (see the Syllabus and Course policies on Canvas).

Directly sharing answers is not okay, but discussing problems with the course staff or with other students is encouraged. Acknowledge any use of an AI system such as ChatGPT or Copilot.

You should start early so that you have time to get help if you're stuck. The drop-in office hours schedule can be found on Canvas. You can also post questions or start discussions on Ed Discussion. The assignment may look long at first glance, but the problems are broken up into steps that should help you to make steady progress.

## Submission

Submit your assignment as a pdf file on Gradescope, and as a notebook (.ipynb) on Canvas. You can access Gradescope through Canvas on the left-side of the class home page. The problems in each homework assignment are numbered. Note: When submitting on Gradescope, please select the correct pages of your pdf that correspond to each problem. This will allow graders to more easily find your complete solution to each problem.

To produce the .pdf, please convert to html and then print to pdf. (You may want to use your pdf print menu to scale the pages to be sure that cells are not truncated.) To convert to html, you can use this [converter notebook](#).

## Topics

- Variational autoencoders (VAEs)
- Undirected graphs
- The graphical lasso

## Help Acknowledgement

I used GPT 5 for help with the coding, mostly for efficiency/readability. I also benefitted greatly from seeing Levon's code, and of course his pretrained model saved me a ton of time.

# Problem 1: Face time (35 points)

In this problem, we will implement a "shoestring" version of [this amazing fake face generator](#), using a variational autoencoder (VAE). Building a generator like the one featured in the article can take a tremendous amount of computational resources, time, and parameter tuning. In this problem we will build a basic version to illustrate the main concepts, and help you to become more familiar with VAEs. Here is an outline of the process that we'll step you through:

## Problem outline:

- Load data
- Create face groups based on attributes
- Construct the VAE
- Define the loss function and train the VAE (Problem 1.1)
- Encode and reconstruct faces (Problem 1.2)
- Visualize the latent space (Problem 1.3)
- Morph between faces (Problem 1.4)
- Shift attributes of faces (Problem 1.5)
- Generate new faces (Problem 1.6)
- Analyze the effect of the scaling factor in the loss function (Problem 1.7, optional)

In the next cell we load the packages that we'll need. If you don't have one or more of these, you can install them with `!pip install <package_name>` in the cell, or outside the notebook with `conda install -c conda-forge <package_name>`

Below, I've imported the starter code from Levon's pretrained model. Thanks Levon :)

```
In [1]: import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
import os
import glob
import pandas as pd
import random
import numpy as np
import imageio
from tqdm import tqdm
from PIL import Image
from sklearn.model_selection import train_test_split
from scipy.stats import norm
import numpy as np
from pathlib import Path
from sklearn.covariance import GraphicalLasso, GraphicalLassoCV
import seaborn as sns
```

```
In [2]: import tensorflow as tf
print(tf.executing_eagerly()) # must be True
```

True

```
In [3]: from google.colab import drive  
  
drive.mount('/content/drive')
```

Mounted at /content/drive

## Loading the data

```
In [4]: # Change these path names to correspond with your directory  
DATASET_PATH = "/content/drive/MyDrive/lfw-deepfunneled"  
ATTRIBUTES_PATH = "/content/drive/MyDrive/lfw_attributes.txt"
```

```
In [5]: # Make sure the above path is correct before running this cell  
dataset = []  
for i, path in enumerate(glob.iglob(os.path.join(DATASET_PATH, "**", "*.jpg")):  
    if i > 1000:  
        break  
    person = path.split("/")[-2]  
    dataset.append({"person": person, "path": path})  
  
dataset = pd.DataFrame(dataset)  
dataset = dataset.groupby("person").filter(lambda x: len(x) < 100 )  
dataset.head(10)
```

```
Out [5]:
```

|   | person               | path                                              |
|---|----------------------|---------------------------------------------------|
| 0 | Jim_OBrien           | /content/drive/MyDrive/lfw-deepfunneled/Jim_OB... |
| 1 | Jim_OBrien           | /content/drive/MyDrive/lfw-deepfunneled/Jim_OB... |
| 2 | Jim_OBrien           | /content/drive/MyDrive/lfw-deepfunneled/Jim_OB... |
| 3 | Stacey_Dales-Schuman | /content/drive/MyDrive/lfw-deepfunneled/Stacey... |
| 4 | Steve_Karsay         | /content/drive/MyDrive/lfw-deepfunneled/Steve_... |
| 5 | Ernest_Hollings      | /content/drive/MyDrive/lfw-deepfunneled/Ernest... |
| 6 | Jenny_Romero         | /content/drive/MyDrive/lfw-deepfunneled/Jenny_... |
| 7 | Shaun_Rusling        | /content/drive/MyDrive/lfw-deepfunneled/Shawn_... |
| 8 | Beatrice_Dalle       | /content/drive/MyDrive/lfw-deepfunneled/Beatri... |
| 9 | Alejandro_Toledo     | /content/drive/MyDrive/lfw-deepfunneled/Alejan... |

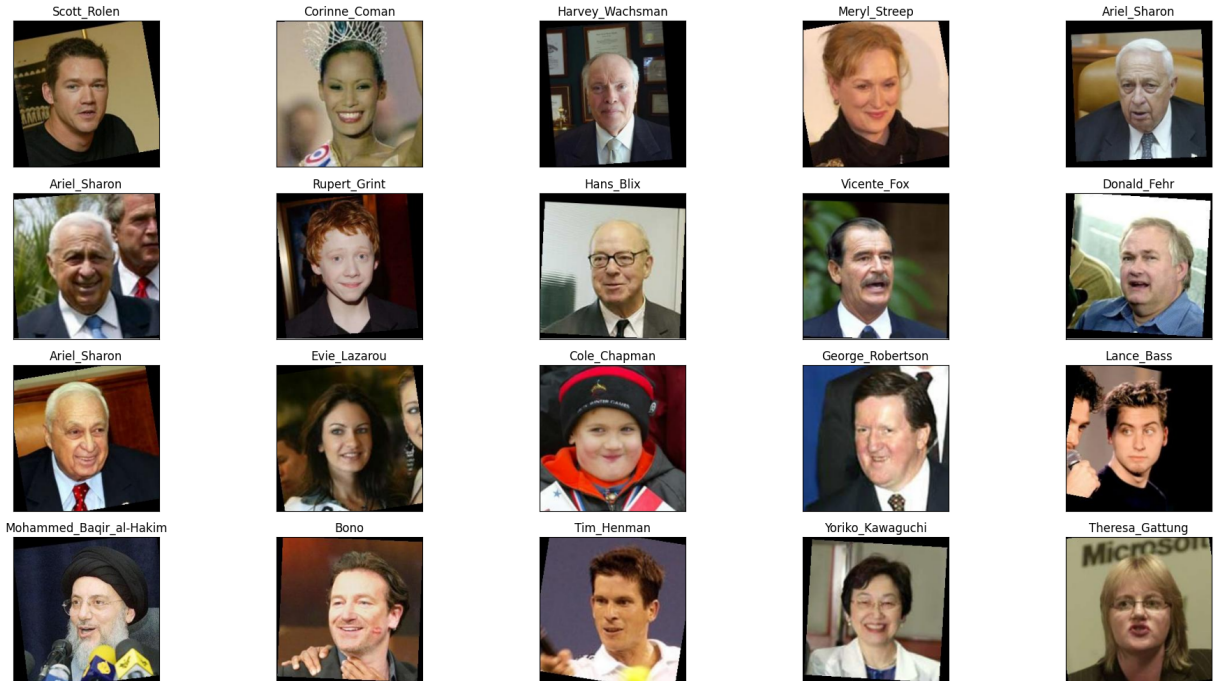
The following cell will display some sample images

```
In [6]: sampled_id = []  
  
plt.figure(figsize=(20,10))  
for i in range(20):  
    idx = random.randint(0, len(dataset))  
    img = plt.imread(dataset.path.iloc[idx])
```

```

plt.subplot(4, 5, i+1)
plt.imshow(img)
plt.title(dataset.person.iloc[idx])
plt.xticks([])
plt.yticks([])
sampled_id.append(idx)
plt.tight_layout()
plt.show()

```



```

In [7]: def fetch_dataset(dx=70, dy=70, dimx=45, dimy=45, limit=500):
        """
        Load and preprocess a subset of the face dataset.

        Parameters
        -----
        dx, dy : int
            Pixel margins to crop from the left/right (dx) and top/bottom (dy).
        dimx, dimy : int
            Output image resolution after resizing.
        limit : int
            Maximum number of images to load (useful for quick testing).

        Returns
        -----
        all_photos : np.ndarray
            Array of processed images with shape (N, dimx, dimy, 3).
        all_attrs : pd.DataFrame
            Corresponding attribute values for each image.
        """

        # 1. Load attributes
        df_attrs = pd.read_csv(ATTRIBUTES_PATH, sep='\t', skiprows=1)
        df_attrs = pd.DataFrame(df_attrs.iloc[:, :-1].values, columns=df_attrs.c

        # 2. Build photo list

```

```

records = []
for img_path in Path(DATASET_PATH).rglob("*.jpg"):
    fname = img_path.stem.replace("_", " ")
    parts = fname.split()
    person = " ".join(parts[:-1])
    imagenum = int(parts[-1])
    records.append({"person": person, "imagenum": imagenum, "photo_path":

photo_ids = pd.DataFrame(records)

# 3. Merge attributes with paths
df = pd.merge(df_attrs, photo_ids, on=("person", "imagenum"))
assert len(df) == len(df_attrs), "lost some data when merging dataframes

# Limit number of images
if limit is not None:
    df = df.iloc[:limit].copy()

# 4. Vectorized image processing
def process_image(path):
    with Image.open(path) as im:
        im = im.convert("RGB")
        w, h = im.size
        im = im.crop((dx, dy, w - dx, h - dy))
        im = im.resize((dimx, dimy), Image.LANCZOS)
        return np.array(im, dtype=np.uint8)

all_photos = np.stack([process_image(p) for p in df["photo_path"]])
all_attrs = df.drop(columns=["photo_path", "person", "imagenum"])

return all_photos, all_attrs

```

```

In [8]: data, attrs = fetch_dataset()

data = np.array(data/255, dtype='float32')

```

```

In [9]: def plot_gallery(images, h, w, n_row=3, n_col=6, with_title=False, titles=[]
plt.figure(figsize=(1.75 * n_col, 2 * n_row))
plt.subplots_adjust(bottom=0, left=.01, right=.99, top=.90, hspace=.35)
for i in range(n_row * n_col):
    plt.subplot(n_row, n_col, i + 1)
    try:
        plt.imshow(images[i].reshape((h, w, 3)), cmap=plt.cm.gray, vmin=
        if with_title:
            plt.title(titles[i])
        plt.xticks(())
        plt.yticks(())
    except:
        pass

```

## Create Face Groups

```

In [10]: IMAGE_H = data.shape[1]
IMAGE_W = data.shape[2]

```

```
N_CHANNELS = 3
```

```
In [11]: smile_ids = attrs['Smiling'].sort_values(ascending=False).head(36).index.values
smile_data = data[smile_ids]

no_smile_ids = attrs['Smiling'].sort_values(ascending=True).head(36).index.values
no_smile_data = data[no_smile_ids]

eyeglasses_ids = attrs['Eyeglasses'].sort_values(ascending=False).head(36).index.values
eyeglasses_data = data[eyeglasses_ids]

sunglasses_ids = attrs['Sunglasses'].sort_values(ascending=False).head(36).index.values
sunglasses_data = data[sunglasses_ids]

mustache_ids = attrs['Mustache'].sort_values(ascending=False).head(36).index.values
mustache_data = data[mustache_ids]

male_ids = attrs['Male'].sort_values(ascending=False).head(36).index.values
male_data = data[male_ids]

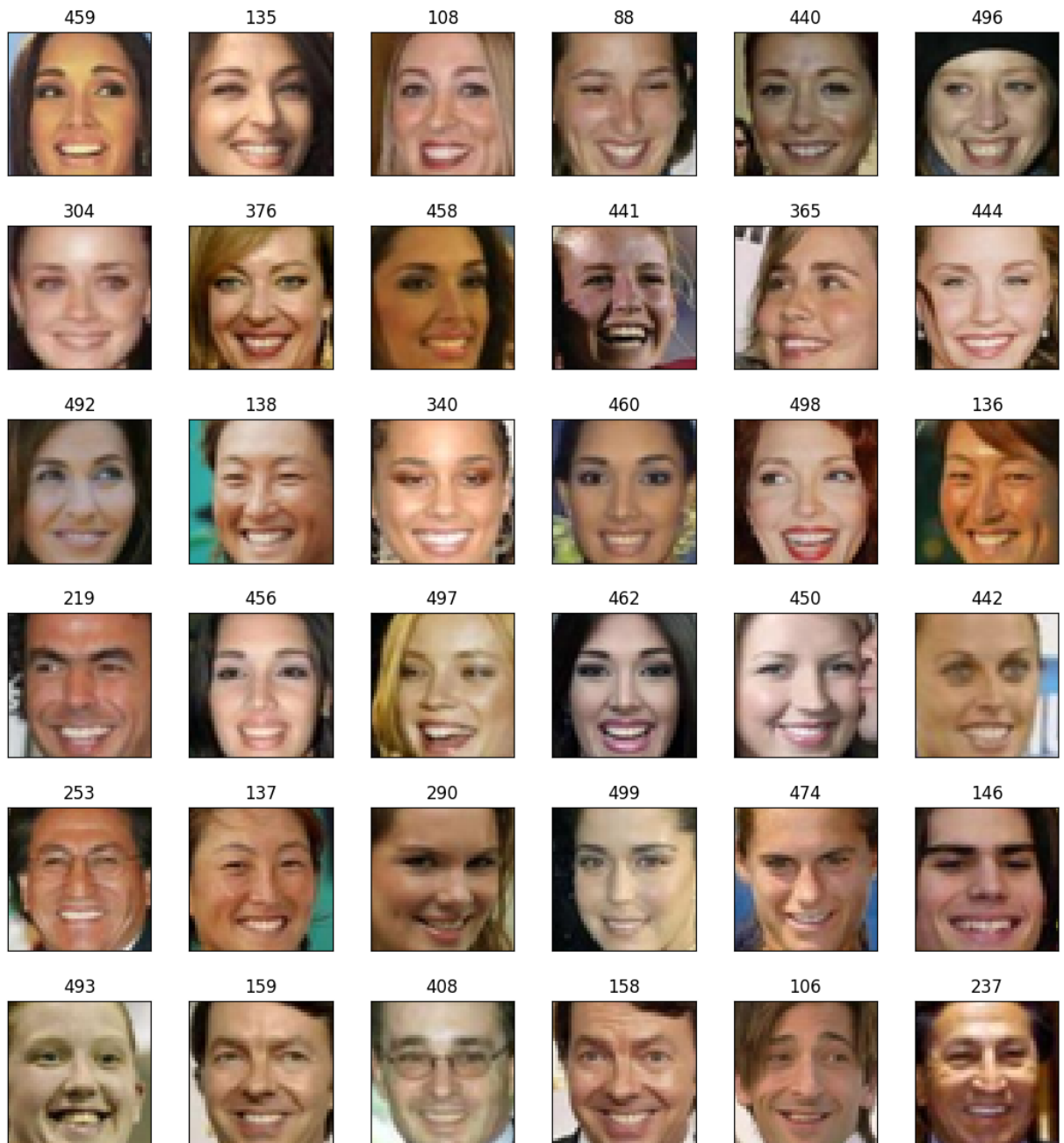
female_ids = attrs['Male'].sort_values(ascending=True).head(36).index.values
female_data = data[female_ids]

eyeclosed_ids = attrs['Eyes Open'].sort_values(ascending=True).head(36).index.values
eyeclosed_data = data[eyeclosed_ids]

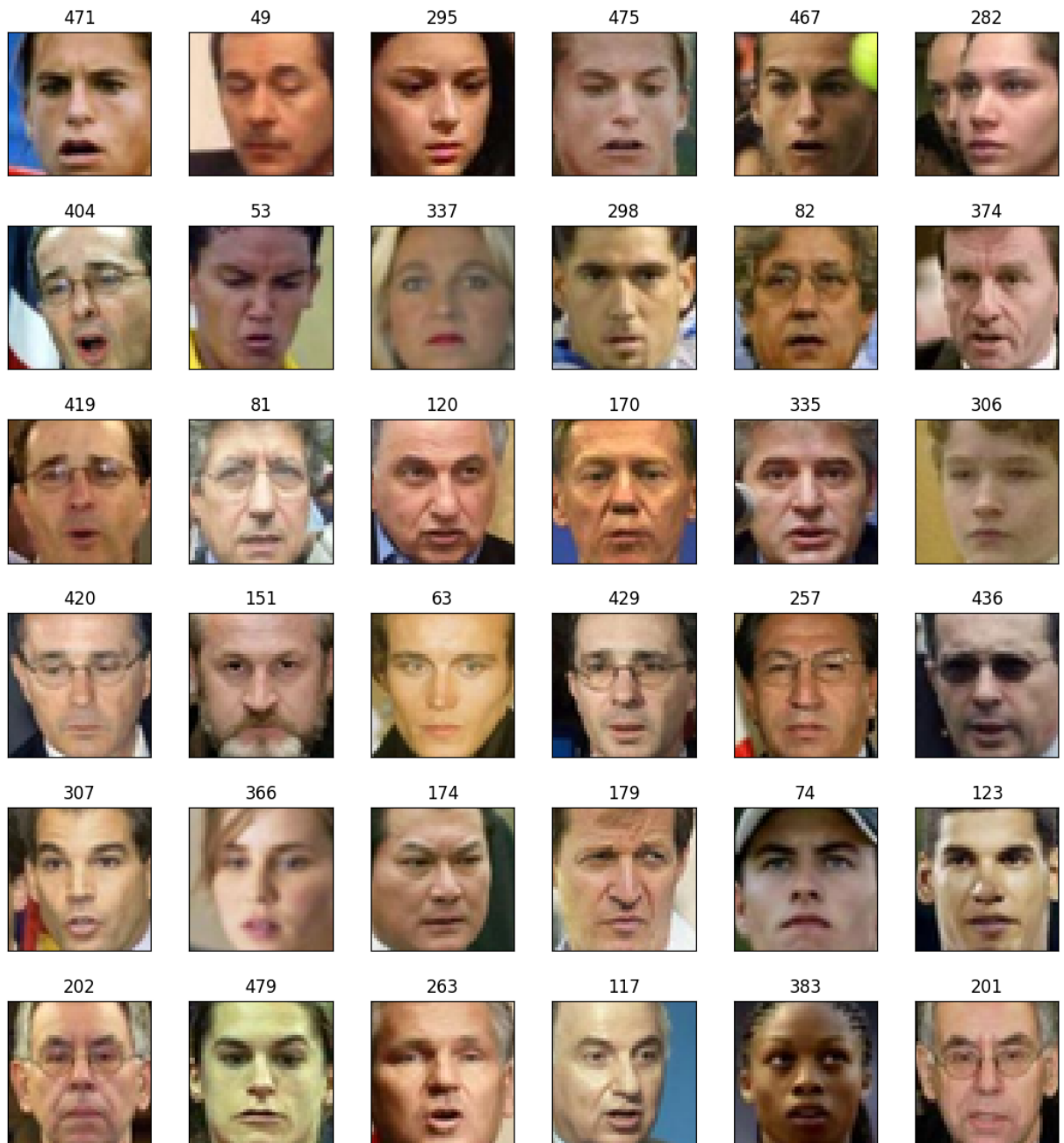
mouthopen_ids = attrs['Mouth Wide Open'].sort_values(ascending=False).head(36).index.values
mouthopen_data = data[mouthopen_ids]

makeup_ids = attrs['Heavy Makeup'].sort_values(ascending=False).head(36).index.values
makeup_data = data[makeup_ids]
```

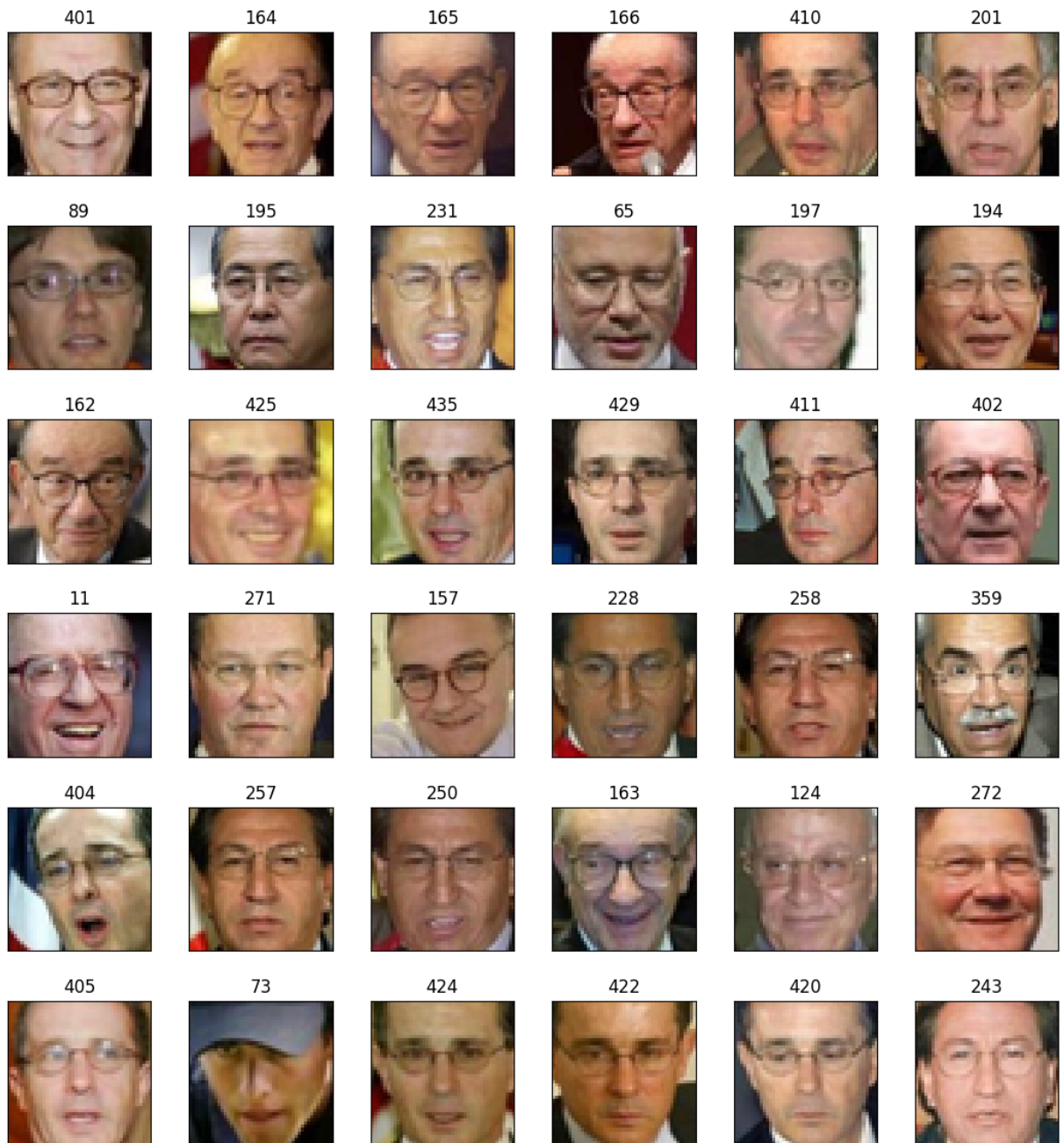
```
In [12]: plot_gallery(smile_data, IMAGE_H, IMAGE_W, n_row=6, n_col=6, with_title=True)
```



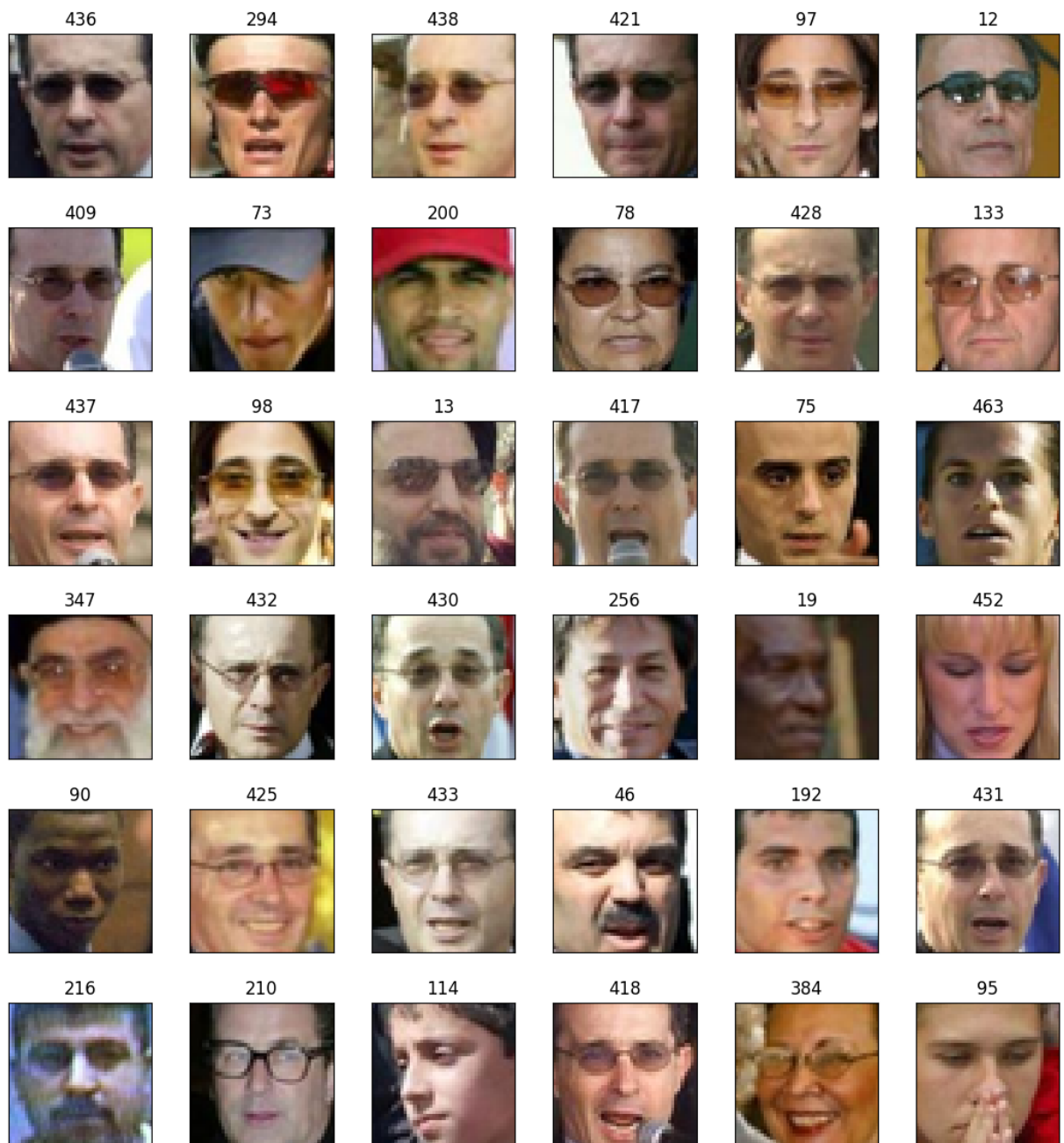
```
In [13]: plot_gallery(no_smile_data, IMAGE_H, IMAGE_W, n_row=6, n_col=6, with_title=T
```



```
In [14]: plot_gallery(eyeglasses_data, IMAGE_H, IMAGE_W, n_row=6, n_col=6, with_title
```



```
In [15]: plot_gallery(sunglasses_data, IMAGE_H, IMAGE_W, n_row=6, n_col=6, with_title
```



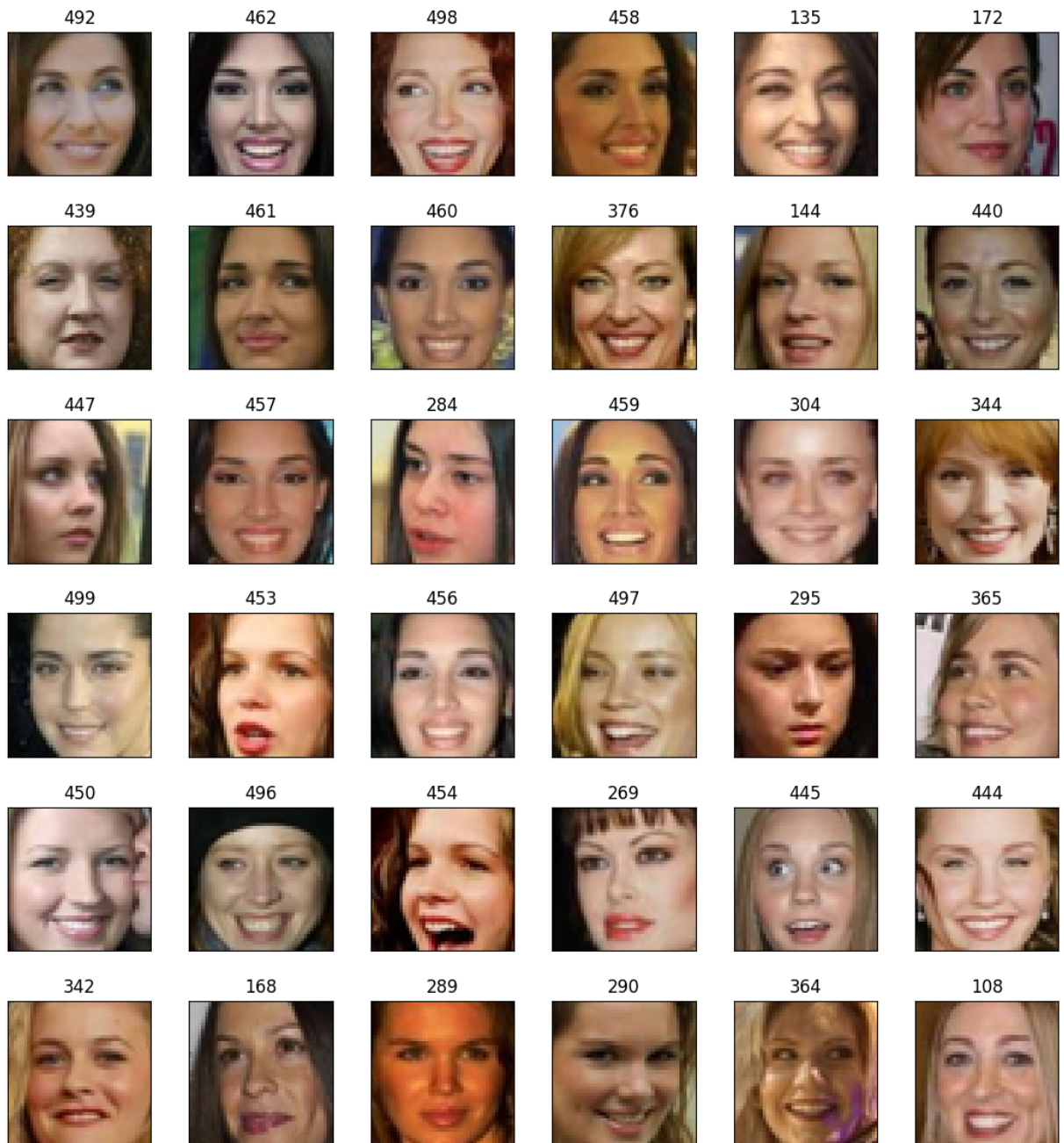
```
In [16]: plot_gallery(mustache_data, IMAGE_H, IMAGE_W, n_row=6, n_col=6, with_title=T
```



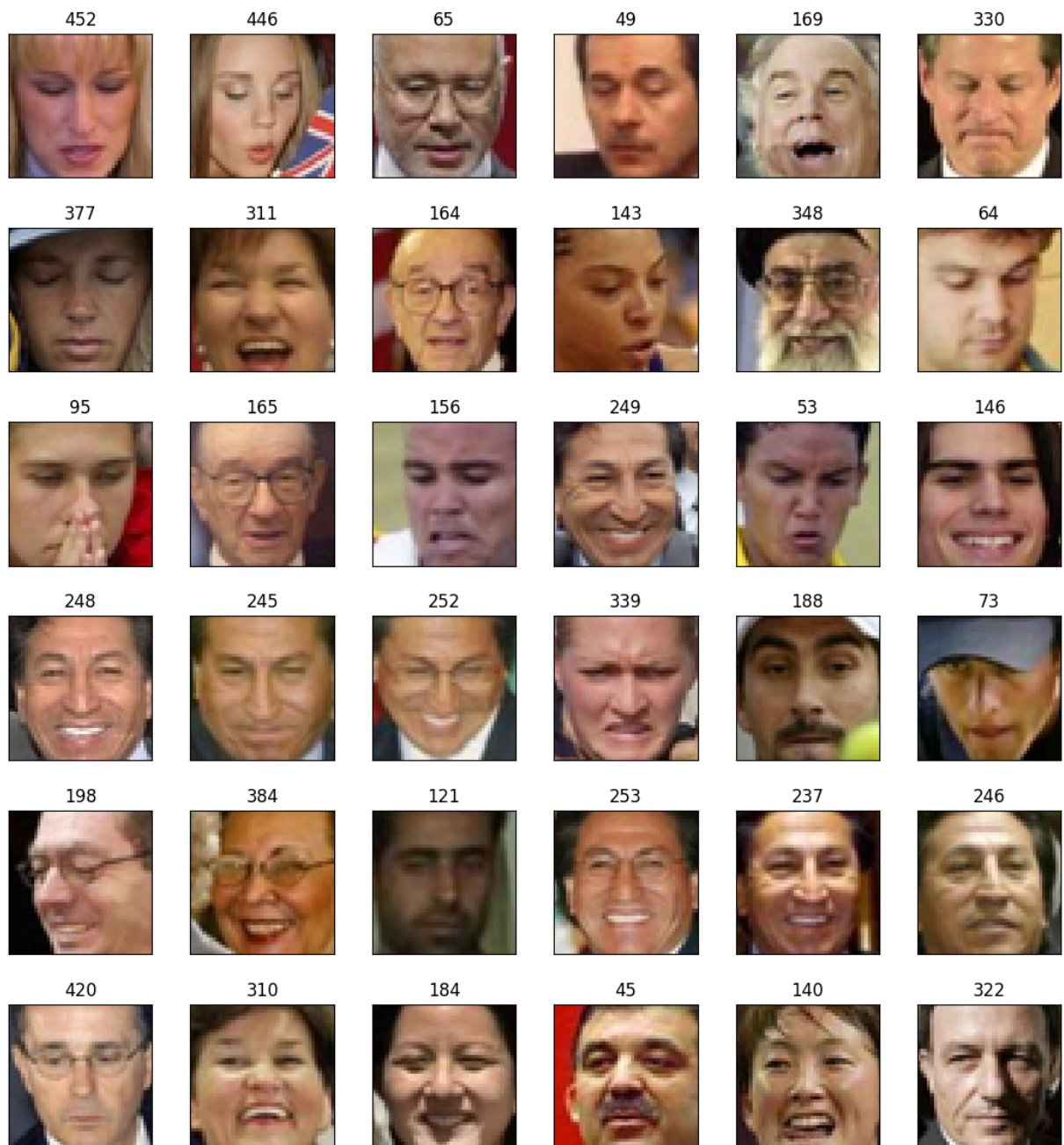
```
In [17]: plot_gallery(male_data, IMAGE_H, IMAGE_W, n_row=6, n_col=6, with_title=True,
```



```
In [18]: plot_gallery(female_data, IMAGE_H, IMAGE_W, n_row=6, n_col=6, with_title=True)
```



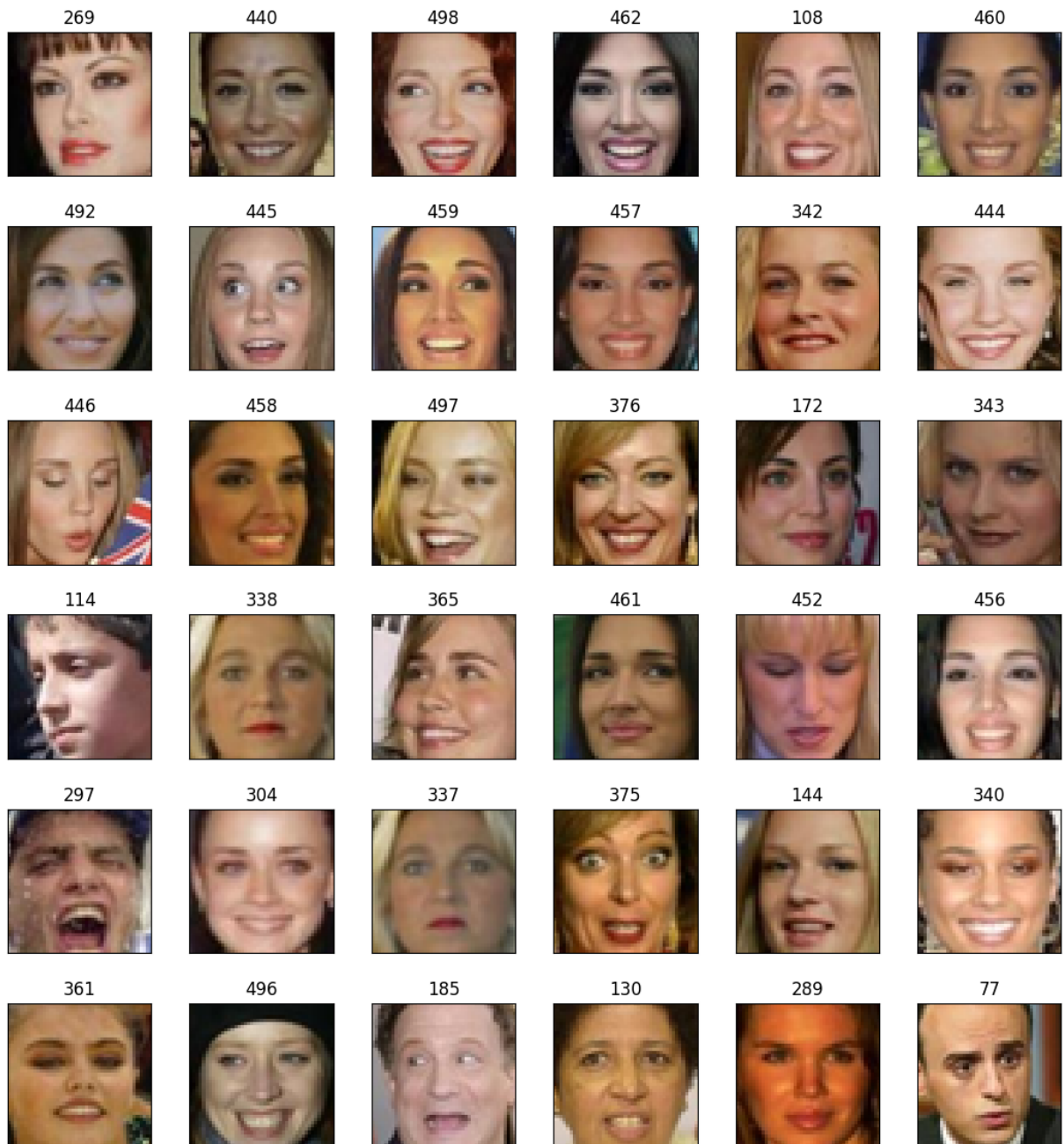
```
In [19]: plot_gallery(eyeclosed_data, IMAGE_H, IMAGE_W, n_row=6, n_col=6, with_title=
```



```
In [20]: plot_gallery(mouthopen_data, IMAGE_H, IMAGE_W, n_row=6, n_col=6, with_title=
```



```
In [21]: plot_gallery(makeup_data, IMAGE_H, IMAGE_W, n_row=6, n_col=6, with_title=True)
```



```
In [22]: X_train, X_val = train_test_split(data, test_size=0.2, random_state=365)
```

```
In [23]: from tensorflow import keras
import tensorflow as tf
import numpy as np

# 1) Lambda function
def sample_latent_features(args):
    mu, logvar = args
    eps = tf.random.normal(shape=tf.shape(mu))
    return mu + tf.exp(0.5 * logvar) * eps

encoder_model = keras.models.load_model(
    "/content/drive/MyDrive/Levon_Encoder.keras",
    compile=False, # ok for inference
    custom_objects={"sample_latent_features": sample_latent_features},
```

```

    safe_mode=False
)

decoder_model = keras.models.load_model(
    "/content/drive/MyDrive/Levon_Decoder.keras",
    compile=False
)

autoencoder = keras.models.load_model(
    "/content/drive/MyDrive/Levon_Autoencoder.keras",
    compile=False,
    custom_objects={"sample_latent_features": sample_latent_features},
    safe_mode=False
)

# print("eager?", tf.executing_eagerly()) # should be True

```

## Constructing the encoder

Recall that the encoder part of the VAE architecture maps a data point to a variational mean and (log) variance. The mean is a point in the latent space.

N.B. I commented out the following code snippets. After trying to run this, it was taking too long, and I began to use Levon's pretrained model instead.

```
In [24]: # LATENT_SPACE_SIZE = 100
```

The "reparameterization trick" draws samples from the variational distribution that are parameterized by the variational mean and variance, so that the parameters of the encoder network can be trained.

```
In [25]: # def sample_latent_features(distribution):
#         distribution_mean, distribution_variance = distribution
#         batch_size = tensorflow.shape(distribution_variance)[0]
#         random = tensorflow.keras.backend.random_normal(shape=(batch_size, ten
#         return distribution_mean + tensorflow.exp(0.5 * distribution_variance)
```

```
In [26]: # input_data = tensorflow.keras.layers.Input(shape=(45, 45, 3))

# encoder = tensorflow.keras.layers.Conv2D(64, (5,5), activation='relu')(inp
# encoder = tensorflow.keras.layers.MaxPooling2D((2,2))(encoder)

# encoder = tensorflow.keras.layers.Conv2D(64, (3,3), activation='relu')(enc
# encoder = tensorflow.keras.layers.MaxPooling2D((2,2))(encoder)

# encoder = tensorflow.keras.layers.Conv2D(32, (3,3), activation='relu')(enc
# encoder = tensorflow.keras.layers.MaxPooling2D((2,2))(encoder)

# encoder = tensorflow.keras.layers.Flatten()(encoder)

# distribution_mean = tensorflow.keras.layers.Dense(LATENT_SPACE_SIZE, name=
# distribution_variance = tensorflow.keras.layers.Dense(LATENT_SPACE_SIZE, n
```

```
# latent_encoding = tensorflow.keras.layers.Lambda(sample_latent_features)([
# encoder_model = tensorflow.keras.Model(input_data, latent_encoding)
# encoder_model.summary()
```

## Construct the decoder

The decoder network in the VAE architecture maps a latent vector to an image. This is done with a series of transposed convolutional layers, since it must map from low to high dimensions.

```
In [27]: # decoder_input = tensorflow.keras.layers.Input(shape=(LATENT_SPACE_SIZE,))
# decoder = tensorflow.keras.layers.Reshape((1, 1, 100))(decoder_input)
# decoder = tensorflow.keras.layers.Conv2DTranspose(64, (3,3), activation='r
# decoder = tensorflow.keras.layers.UpSampling2D((2,2))(decoder)

# decoder = tensorflow.keras.layers.Conv2DTranspose(32, (3,3), activation='r
# decoder = tensorflow.keras.layers.UpSampling2D((2,2))(decoder)

# decoder = tensorflow.keras.layers.Conv2DTranspose(16, (5,5), activation='r
# decoder = tensorflow.keras.layers.UpSampling2D((2,2))(decoder)

# decoder_output = tensorflow.keras.layers.Conv2DTranspose(3, (6,6), activat

# decoder_model = tensorflow.keras.Model(decoder_input, decoder_output)
# decoder_model.summary()
```

```
In [28]: # encoded = encoder_model(input_data)
# decoded = decoder_model(encoded)
# autoencoder = tensorflow.keras.models.Model(input_data, decoded)
```

```
In [29]: # def get_loss(distribution_mean, distribution_variance, factor, batch_size):

#     def get_reconstruction_loss(y_true, y_pred, factor, batch_size):
#         reconstruction_loss = tensorflow.math.squared_difference(y_true, y
#         reconstruction_loss_batch = tensorflow.reduce_sum(reconstruction_l
#         return 0.5*reconstruction_loss_batch*factor

#     def get_kl_loss(distribution_mean, distribution_variance, batch_size):
#         kl_loss = LATENT_SPACE_SIZE + distribution_variance - tensorflow.s
#         kl_loss_batch = tensorflow.reduce_sum(kl_loss)/batch_size
#         return kl_loss_batch*(-0.5)

#     def total_loss(y_true, y_pred):
#         reconstruction_loss_batch = get_reconstruction_loss(y_true, y_pred
#         kl_loss_batch = get_kl_loss(distribution_mean, distribution_varian
#         return reconstruction_loss_batch + kl_loss_batch

#     return total_loss
```

```
In [30]: # from keras import ops, layers, Model
# from keras.losses import mean_squared_error as mse
```

```

# # --- KL layer ---
# class KLLoss(layers.Layer):
#     def call(self, inputs):
#         mu, logvar = inputs
#         # KL per sample:  $-0.5 * \sum(1 + \logvar - \mu^2 - \exp(\logvar))$ 
#         kl_per_sample = -0.5 * ops.sum(1.0 + logvar - ops.square(mu) - ops.exp(logvar), axis=-1)
#         self.add_loss(ops.mean(kl_per_sample)) # scalar KL
#         return inputs # pass-through

# # 1) Create the KL node
# kl_node = KLLoss(name="kl_loss")([distribution_mean, distribution_variance])

# # 2) Tie the KL node into the forward graph but keep output = decoded
# decoded = layers.Lambda(lambda xs: xs[0], name="with_kl")([decoded, kl_node.output])

# # 3) (Re)build the model so the KL loss is included
# autoencoder = tensorflow.keras.models.Model(input_data, decoded, name="autoencoder")

# # 4) reconstruction loss
# def reconstruction_loss(y_true, y_pred, sum_over_channels=False):
#     # per-pixel MSE (averaged over channels)
#     per_pixel = mse(y_true, y_pred) # (B, H, W)
#     H = ops.shape(per_pixel)[1]
#     W = ops.shape(per_pixel)[2]
#     factor = ops.cast(H * W, per_pixel.dtype)
#     if sum_over_channels:
#         C = ops.shape(y_true)[-1] # undo the channel mean
#         factor = factor * ops.cast(C, per_pixel.dtype)
#     return ops.mean(per_pixel) * factor # mean over batch, sum over channels

```

## 1.1 Deriving the loss function (5 points)

Derive the loss function defined in the cell above from the probability model perspective. You can ignore the scalar `factor` in your derivation. Show your work using either LaTeX or a picture of your written solution.

Hint: Think about how the total loss is related to the ELBO.

## 1.1

With the generative model

$$z \sim p(z) = N(0, I), \quad x|z \sim p(x|z; \theta) = N(f(z; \theta), \sigma^2 I)$$

we approximate  $p$  by  $q_\theta(z|x) = N(\mu(x; \theta), \sigma^2; \theta)$

We maximize the ELBO, i.e., minimize

$$-ELBO = \underbrace{-\mathbb{E}_q[\log p(x|z; \theta)]}_{\text{KL Divergence}} + \underbrace{D_{KL}(q_\theta(z|x) || p(z))}_{\text{KL Divergence}}$$

a) Reconstruction Loss:

$$\log p(x|z; \theta) = -\frac{1}{2\sigma^2} \|x - f(z; \theta)\|_2^2 - (\text{constant})$$

$$\Rightarrow -\mathbb{E}_q[\log p(x|z; \theta)] \propto \frac{1}{2\sigma^2} \mathbb{E}_q[\|x - f(z; \theta)\|_2^2]$$

Via reparameterization,  $\|x - f(z; \theta)\|_2^2$  becomes the familiar MSE, which we see in the code as

$$\text{per\_pixel} = \text{mse}(y\_true, y\_pred)$$

[scaled by a factor]

b) KL Divergence

For diagonal Gaussians  $q, p$ :

$$D_{KL}(q||p) = \frac{1}{2} \sum_j^d (\mu_j^2 + \sigma_j^2 - 1 - \log \sigma_j^2) \quad \leftarrow d \text{ is latent dim.}$$

$$= -\frac{1}{2} \sum_j^d (1 + \log \sigma_j^2 - \mu_j^2 - e^{\log \sigma_j^2}),$$

which we see in the code in the `KLloss` class

Finally, we sum the two together to get our (negative) ELBO.

The following three cells train the model. You can just run them. It may take a while to run on your laptop.

```
In [31]: # X_train, X_val = train_test_split(data, test_size=0.2, random_state=365)
```

```
In [32]: # batch_size = 64
# # autoencoder.compile(loss=get_loss(distribution_mean, distribution_varian
# #                                     batch_size = batch_size), optimizer='a
# # autoencoder.compile(optimizer="adam", loss=reconstruction_loss)
```

```
# autoencoder.summary()
```

```
In [33]: # autoencoder.fit(X_train, X_train, epochs=50, batch_size=64, validation_dat
```

```
In [34]: # Save your trained model (for later use)
```

```
# autoencoder.save("My_Trained_VAE")
# encoder_model.save("My_Trained_encoder")
# decoder_model.save("My_Trained_decoder")
```

If you experience issues while training the models above or if the process is taking too long, you may also use our pre-trained versions of the VAE, encoder, and decoder, which are available on Canvas.

No points will be taken off if you decide to use the pre-trained models, but we do recommend trying out the training process above. In case you encounter any issues when loading in the models, please post your questions on Ed discussion.

```
In [35]: # Load Pre-trained Model
```

```
# batch_size = 64
# autoencoder = tensorflow.keras.models.load_model("Trained_VAE", compile=False)
# autoencoder.compile(loss=get_loss(distribution_mean, distribution_variance))

# encoder_model = tensorflow.keras.models.load_model("Trained_encoder", compile=False)
# encoder_model.compile(loss=get_loss(distribution_mean, distribution_variance))

# decoder_model = tensorflow.keras.models.load_model("Trained_decoder", compile=False)
# decoder_model.compile(loss=get_loss(distribution_mean, distribution_variance))
```

## 1.2 Reconstructing faces (3 points)

The following cell encodes and reconstructs 16 random faces from the validation set with the trained VAE. Run the cell and comment on the reconstructed faces. (3 points)

- Do the reconstructed faces resemble the original images? How are they similar/different?
- Are there any faces that are reconstructed better or worse than the others? Can you think of why?
- Comment on any other aspects of your findings that are interesting to you.

```
In [36]: sample_index = random.sample(range(1, len(X_val)), 16)
```

```
fig, axs = plt.subplots(4, 8)
fig.set_figheight(10)
fig.set_figwidth(15)

for i in range(4):
    for j in range(4):
```

```

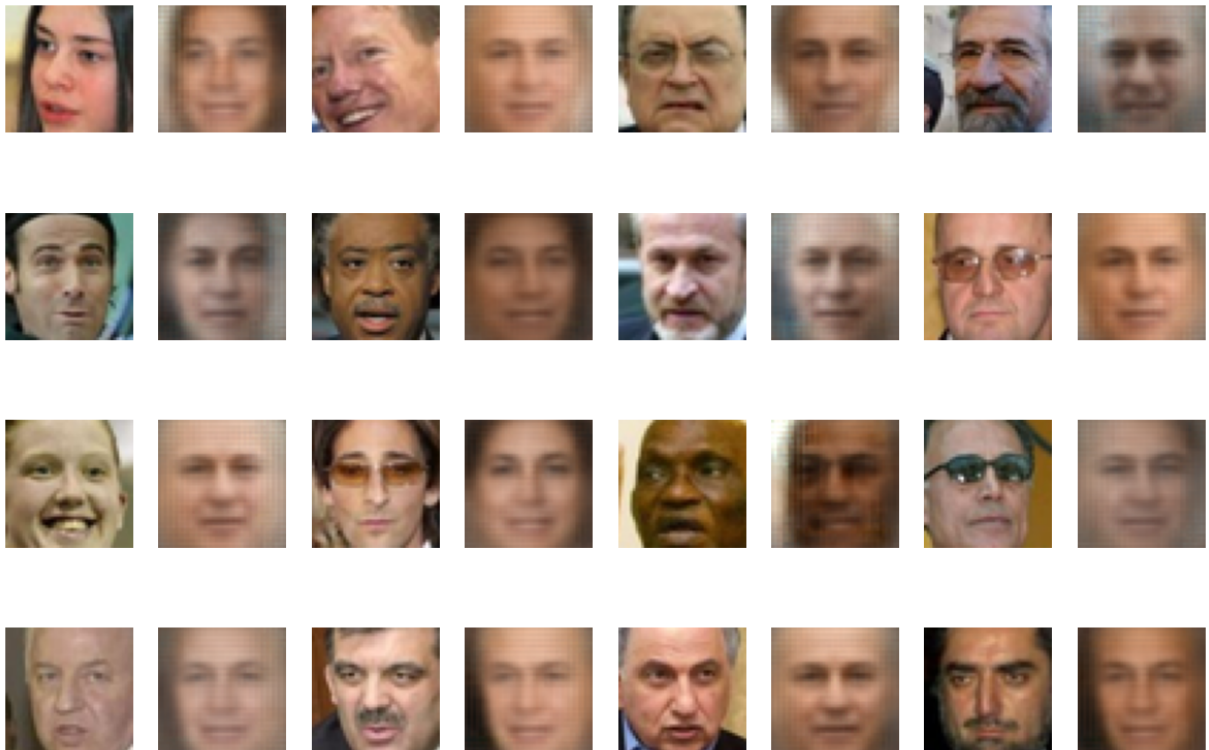
axs[i, 2*j].imshow(X_val[sample_index[4*i+j], :, :, :])
axs[i, 2*j].axis('off')
axs[i, 2*j+1].imshow(np.clip(autoencoder.predict(np.array([X_val[sam
axs[i, 2*j+1].axis('off')

```

```

1/1 ————— 1s 523ms/step
1/1 ————— 0s 41ms/step
1/1 ————— 0s 40ms/step
1/1 ————— 0s 43ms/step
1/1 ————— 0s 39ms/step
1/1 ————— 0s 42ms/step
1/1 ————— 0s 41ms/step
1/1 ————— 0s 53ms/step
1/1 ————— 0s 41ms/step
1/1 ————— 0s 43ms/step
1/1 ————— 0s 41ms/step
1/1 ————— 0s 40ms/step
1/1 ————— 0s 41ms/step
1/1 ————— 0s 42ms/step
1/1 ————— 0s 40ms/step
1/1 ————— 0s 43ms/step

```



## 1.2 Response

Do the reconstructed faces resemble the original images? How are they similar/different? Are there any faces that are reconstructed better or worse than the others? Can you think of why? Comment on any other aspects of your findings that are interesting to you.

The reconstructed faces look alright. It seems like we are able to match the general shape and skin tone of each face pretty well, but many faces miss details from the original. For instance, I can see one that misses a moustache. In another, the eyebrows are not in the same position. Etc. Missing the moustache is especially surprising considering we have a training set for it (although the one in question is quite subtle). In general, missing small details make sense, as the model is trained to generate data covering a distribution of possible features/details. Indeed, it seems like the reconstructed faces are typically pretty similar to each other even when the original faces are different. I think it's interesting that skin tone is so well captured.

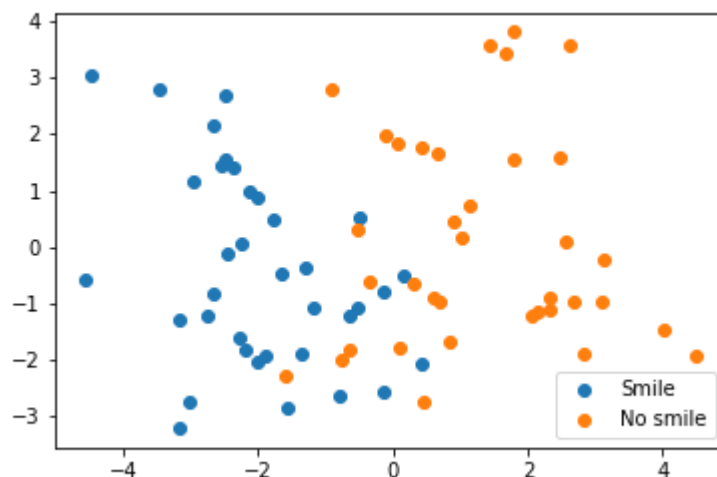
### 1.3 Visualizing the latent space (10 points)

In `vae_demo` from class, the MNIST digits were generated from a two-dimensional latent space. In the current model, the latent space has more than two dimensions, so to visualize it we need to use a dimensionality reduction technique. (If you are not familiar with PCA, please refer to the material for Week 7 of [iML](#).)

In this problem, you will first implement the function `LatentSpace_2D`. (6 points)

1. Calculate the latent space encodings for two sets of faces that are different in one attribute, e.g. smile vs. no smile.
2. Use PCA to reduce the dimension of the latent space codes to two.
3. Visualize the latent space after dimensionality reduction with a scatter plot. Clearly color-code and label the two different groups.

Here is an example using `smile_data` and `no_smile_data`.



Visualize the latent space for at least three pairs of face groups including smile vs. no smile. Comment on how the scatter plots look.

- Are the two groups separable in the two-dimensional latent space? Is this what you

expected? Why or why not? (2 points)

- How do the plots for the three different attributes differ from each other? (2 points)

```
In [37]: def LatentSpace_2D(encoder_model, data1, label1, data2, label2):

    # Extract means from encoder_model
    z_mu1, z_logvar = encoder_model.predict(data1 )
    z_mu2, z_logvar = encoder_model.predict(data2 )

    z_mu_cat = np.concatenate([z_mu1, z_mu2], axis=0)

    # PCA for 2 components
    pca = PCA(n_components=2)
    z_pca = pca.fit_transform(z_mu_cat)

    # Labels
    N1 = z_mu1.shape[0]
    N2 = z_mu2.shape[0]

    labels_1 = np.zeros(N1, dtype=int)
    labels_2 = np.ones(N2, dtype=int)

    all_labels = np.concatenate([labels_1, labels_2], axis=0)

    # Plotting
    plt.figure(figsize=(12, 10))

    scatter = plt.scatter(
        z_pca[:, 0],    # PC 1
        z_pca[:, 1],    # PC2
        c=all_labels,
        cmap='coolwarm',
        alpha=0.6,
        s=15
    )

    from sklearn.metrics import silhouette_score

    score = silhouette_score(z_pca, all_labels)

    legend_handles = [
        plt.Line2D([0], [0], marker='o', color='w', label=label1,
                    markerfacecolor=scatter.to_rgba(0), markersize=10),
        plt.Line2D([0], [0], marker='o', color='w', label=label2,
                    markerfacecolor=scatter.to_rgba(1), markersize=10)
    ]

    plt.legend(handles=legend_handles, title="Dataset Labels", loc="upper ri

    plt.title('VAE Latent Space Visualization (PCA Reduced): {} vs. {}'.form
    plt.xlabel(f'Principal Component 1 ({pca.explained_variance_ratio_[0]:.2
    plt.ylabel(f'Principal Component 2 ({pca.explained_variance_ratio_[1]:.2

    plt.grid(True, linestyle='--', alpha=0.5)
    plt.tight_layout()
```

```
plt.show()
```

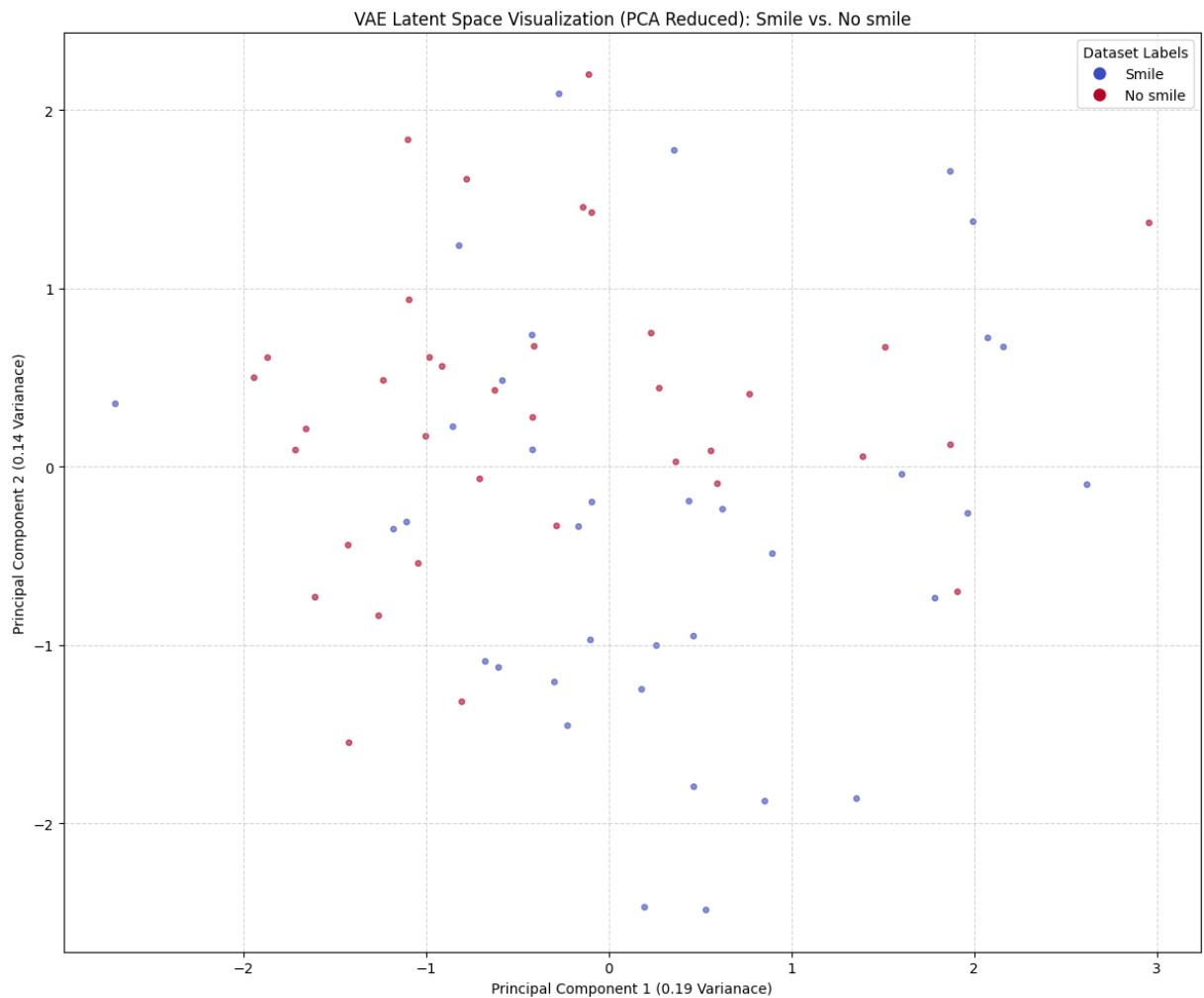
```
In [38]: # Extract the mean and log(sigma^2) layers from the encoder
mu_layer = encoder_model.get_layer("variational_mean")
logvar_layer = encoder_model.get_layer("variational_log_variance")

mu_logvar_model = tf.keras.Model(
    inputs=encoder_model.input,
    outputs=[mu_layer.output, logvar_layer.output],
    name="encoder_mu_logvar_extractor"
)
```

```
In [39]: LatentSpace_2D(mu_logvar_model, smile_data, 'Smile', no_smile_data, 'No smile')
```

```
2/2 ————— 0s 97ms/step
```

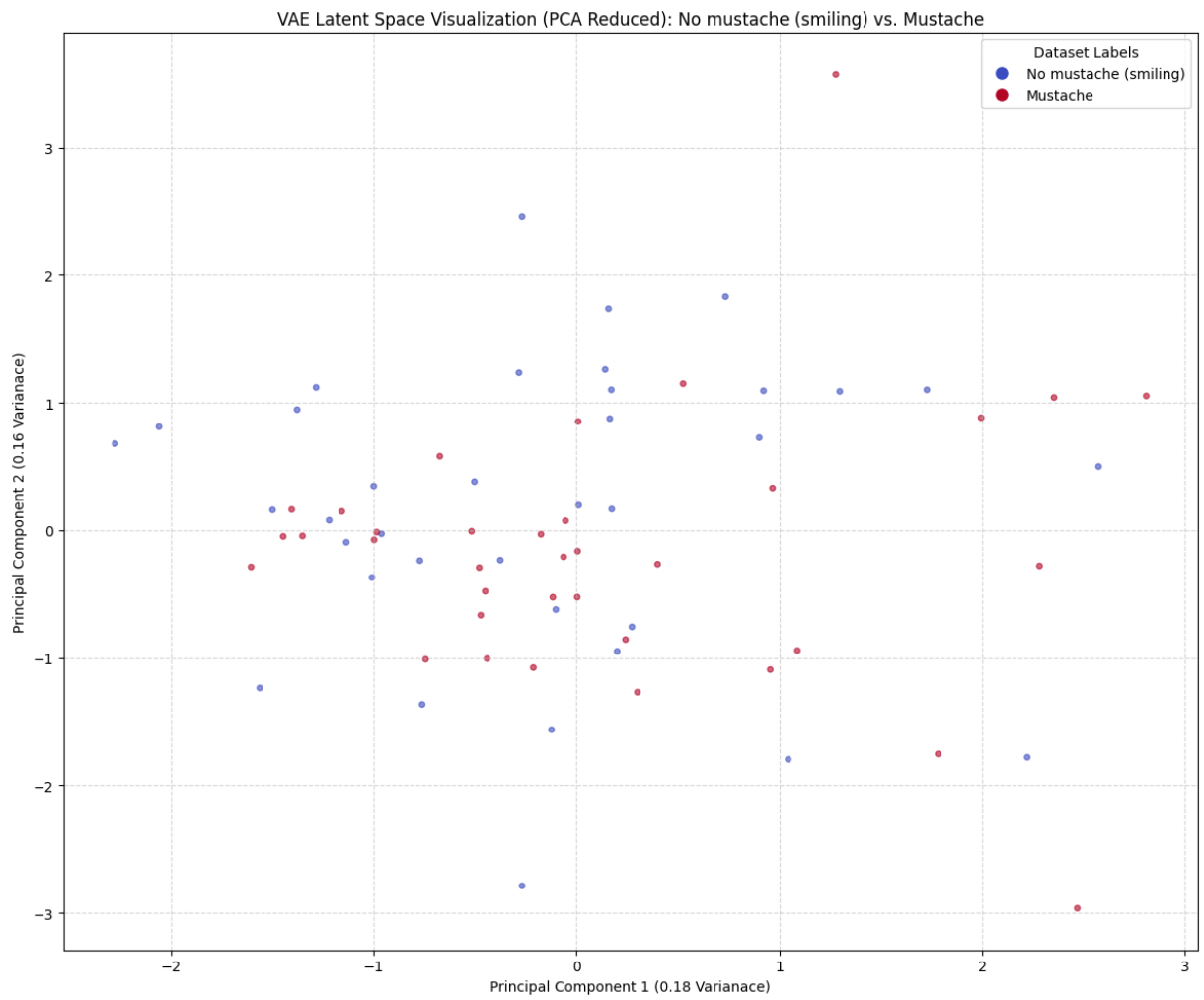
```
2/2 ————— 0s 26ms/step
```



```
In [40]: LatentSpace_2D(mu_logvar_model, smile_data, 'No mustache (smiling)', mustache_d
```

```
2/2 ————— 0s 28ms/step
```

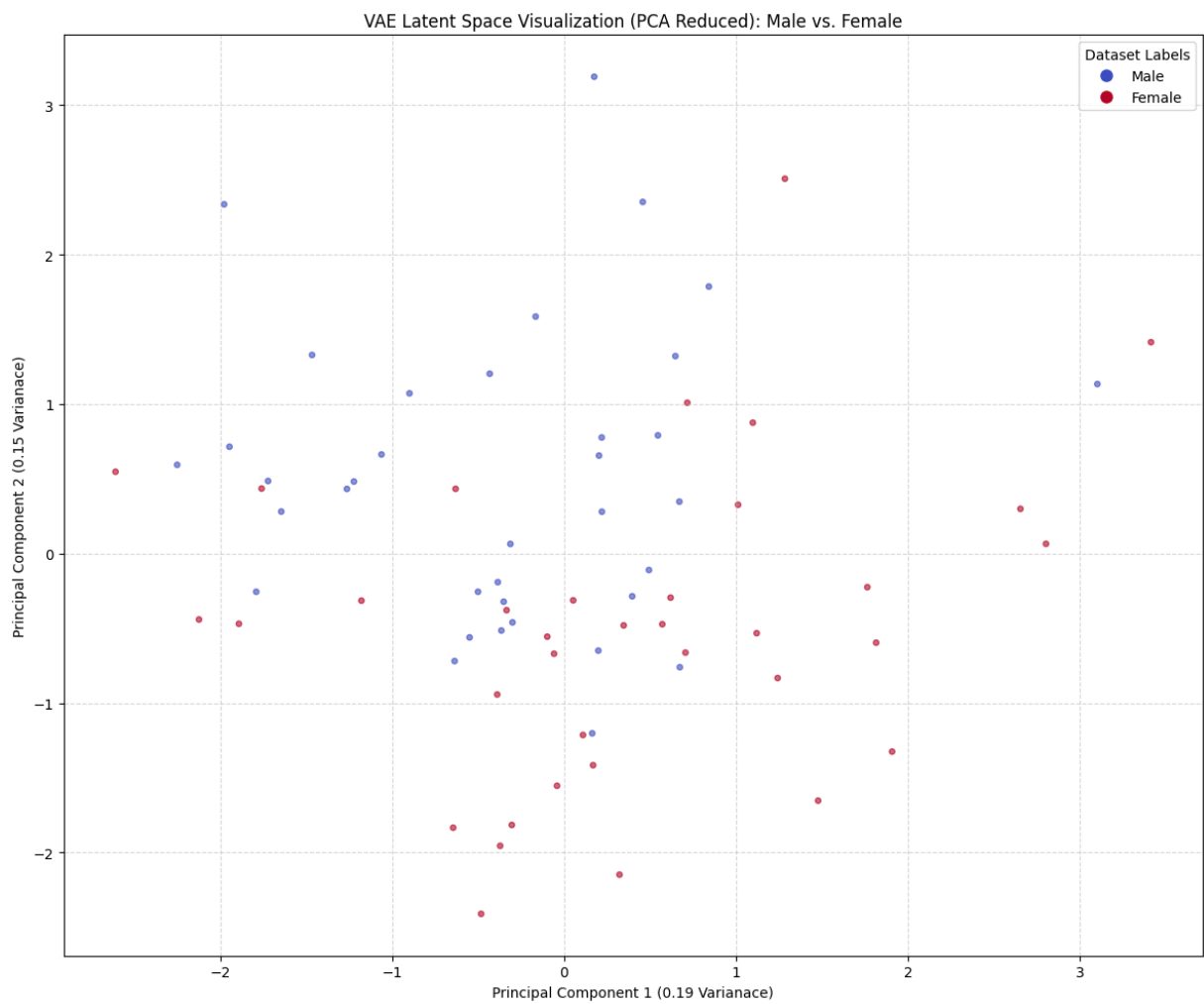
```
2/2 ————— 0s 26ms/step
```



```
In [41]: LatentSpace_2D(mu_logvar_model,male_data,'Male',female_data,'Female')
```

2/2 ===== 0s 26ms/step

2/2 ===== 0s 26ms/step



## 1.3 Response

Are the two groups separable in the two-dimensional latent space? Is this what you expected? Why or why not? How do the plots for the three different attributes differ from each other?

In each of the three groups, I think we do get a bit of separability, but we see it mostly in the male vs. female comparison. The smile vs. no smile group has the most overlap (i.e., hardest to distinguish with two components). We see that maybe a low value of PC2 is associated with a smile, but mostly the two distributions look mixed. For mustache vs. no mustache, the two groups again mostly overlap across both dimensions, with perhaps the no mustache group getting higher values of PC2 on average. Male vs. female seems like the best differentiation, where females tend to receive higher values of PC1 and lower values of PC2. Still though, we see plenty of overlap. Overall, these results don't seem entirely surprising to me, since faces contain many attributes and subtle features; in addition, we saw that the generated faces earlier were quite similar to each other, even when the original faces were different.

## 1.4 Morphing between faces (4 points)

Morph at least 5 pairs of faces with the function `morphBetweenImages` and comment on what you observe.

- Briefly explain how the morphing works. (2 points)
- Do the generated faces look like what you expected? Does any of the pairs work better than the others? If so, what kind of image pairs work better? (2 points)

```
In [42]: # Don't change the function
def morphBetweenImages(img1, img2, num_of_morphs):
    alpha = np.linspace(0,1,num_of_morphs)
    z1 = encoder_model.predict(np.array([img1]))
    z2 = encoder_model.predict(np.array([img2]))
    fig = plt.figure(figsize=(30,5))

    ax = fig.add_subplot(1, num_of_morphs+2, 1)
    ax.imshow(img1)
    ax.axis('off')
    ax.set_title(loc='center', label='original image 1',fontsize=10)

    for i in range(num_of_morphs):
        z = z1*(1-alpha[i]) + z2*alpha[i]
        new_img = decoder_model.predict(z)

        ax = fig.add_subplot(1, num_of_morphs+2, i+2)
```

```

ax.imshow(np.clip(new_img.squeeze(),0,1))
ax.axis('off')
ax.set_title(loc='center', label='alpha={:.2f}'.format(alpha[i]))

ax = fig.add_subplot(1, num_of_morphs+2, num_of_morphs+2)
ax.imshow(img2)
ax.axis('off')
ax.set_title(loc='center', label='original image 2',fontsize=10)
return

```

```

In [43]: sample_index = random.sample(range(1, len(data)), 2)
morphBetweenImages(data[sample_index[0]],data[sample_index[1]],10)

```

```

1/1 _____ 0s 113ms/step
1/1 _____ 0s 40ms/step
1/1 _____ 0s 226ms/step
1/1 _____ 0s 38ms/step
1/1 _____ 0s 41ms/step
1/1 _____ 0s 41ms/step
1/1 _____ 0s 42ms/step
1/1 _____ 0s 40ms/step
1/1 _____ 0s 44ms/step
1/1 _____ 0s 40ms/step
1/1 _____ 0s 40ms/step
1/1 _____ 0s 44ms/step

```



```

In [44]: sample_index = random.sample(range(1, len(data)), 2)
morphBetweenImages(data[sample_index[0]],data[sample_index[1]],10)

```

```

1/1 _____ 0s 39ms/step
1/1 _____ 0s 42ms/step
1/1 _____ 0s 40ms/step
1/1 _____ 0s 43ms/step
1/1 _____ 0s 39ms/step
1/1 _____ 0s 39ms/step
1/1 _____ 0s 40ms/step
1/1 _____ 0s 40ms/step
1/1 _____ 0s 40ms/step
1/1 _____ 0s 40ms/step
1/1 _____ 0s 48ms/step
1/1 _____ 0s 39ms/step

```



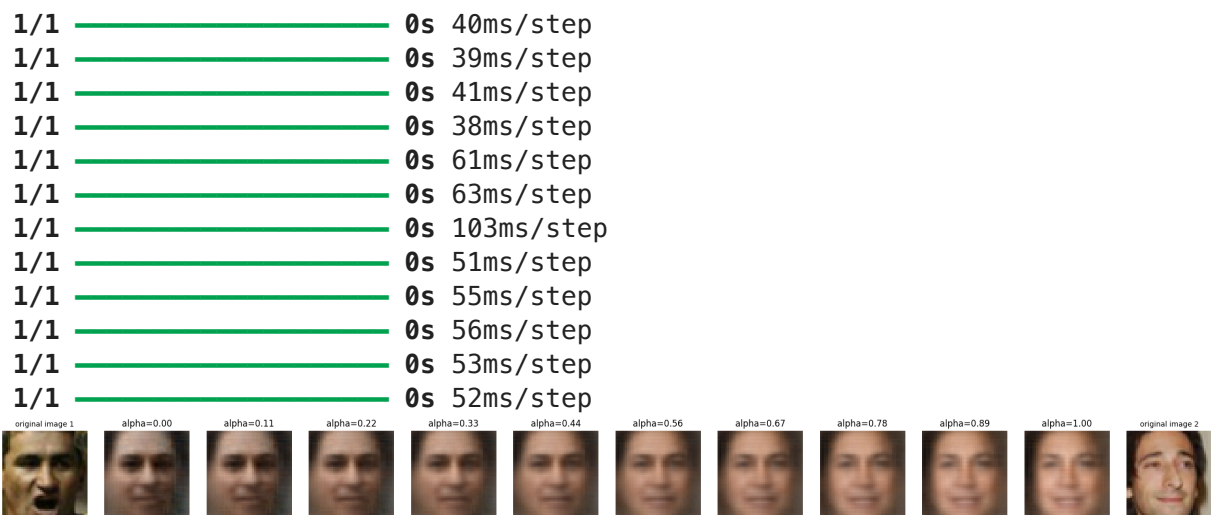
```

In [45]: sample_index = random.sample(range(1, len(data)), 2)
morphBetweenImages(data[sample_index[0]],data[sample_index[1]],10)

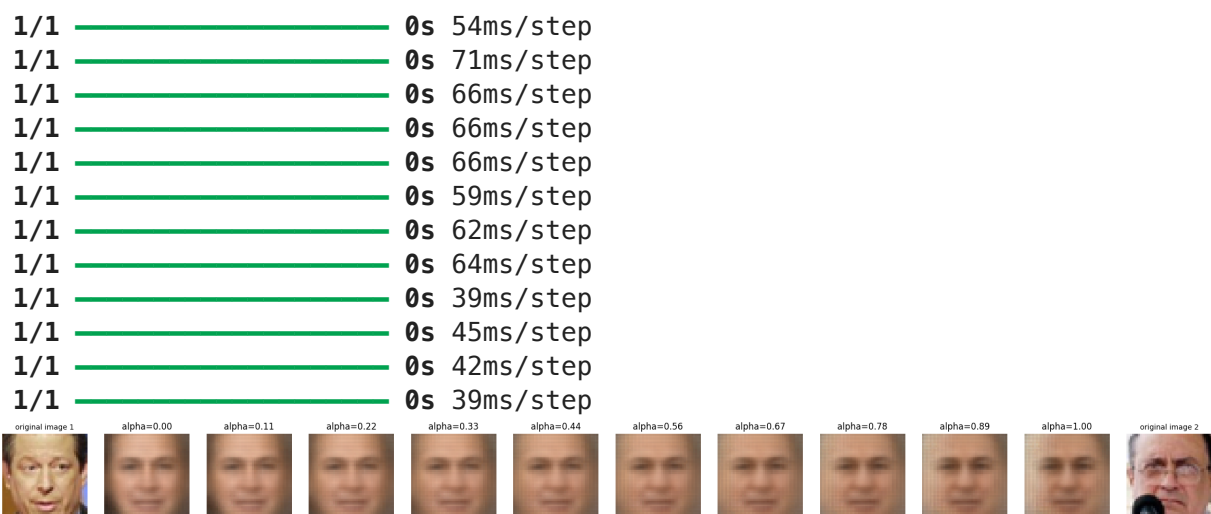
```



```
In [46]: sample_index = random.sample(range(1, len(data)), 2)
morphBetweenImages(data[sample_index[0]],data[sample_index[1]],10)
```



```
In [47]: sample_index = random.sample(range(1, len(data)), 2)
morphBetweenImages(data[sample_index[0]],data[sample_index[1]],10)
```



## 1.4 Response

**Briefly explain how the morphing works. Do the generated faces look like what you expected? Does any of the pairs work better than the others? If so, what kind of image pairs work better?**

The morphing works by interpolating in the latent space of the VAE. Here, both images are represented as low dimensional vectors. To morph, we take a linear combination of those vectors where the weight is controlled by alpha.

The images look... okay. Again, because it seems like we didn't do such an amazing job of capturing the latent space in the first place, interpolating on that latent space is also going to be tough. That being said, we do see that the overall space with low alpha corresponds more to the first image and vice-versa with high alpha. For instance, when the faces are facing different directions, we see that the interpolations gradually change directions as well. We also see that smiles turn a bit more serious, etc. I think image pairs that are more different actually make this method work a bit better, surprisingly to me. I think this is because there is a bit more "easy" variation to get correct. For example, one morphing goes from a light face to a dark face pretty well.

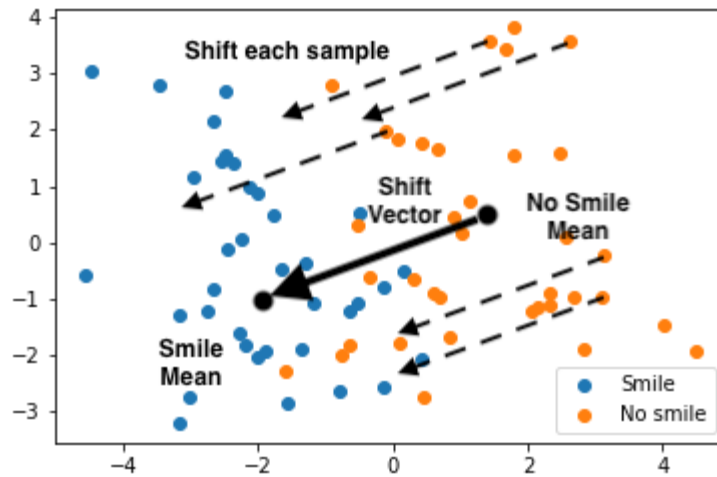
## 1.5 Attribute shift (10 points)

In 1.3, we've seen that faces with the same attributes form clusters in the latent space. In this problem, you will implement a function `AttributeShift` to change one attribute of the faces.

First implement the function `AttributeShift`. (5 points)

1. Calculate the latent space codes for two sets of faces that are different in one attribute, e.g. smile vs. no smile.
2. Calculate the mean latent space code for each group.
3. Get the attribute shifting vector by taking the difference between the two codes.
4. Perform attribute shift by adding the attribute shifting vector to the latent space code of the faces you want to modify.
5. Generate the image using the new latent space codes.

Here is a diagram demonstrating the shift in the latent space. Please note that the two-dimensional latent space is just for demonstration purpose. You should *not* use PCA in this problem. Instead, use the original latent space.



Perform attribute shift on at three attributes including smile. Comment on the faces with shifted attributes. (5 points)

- Do the generated faces look like what you expected? If not, can you think of some possible reasons?
- Do the faces with new attributes resemble the original faces? If not, can you think of some possible reasons?
- Which of the attribute shift is more successful? What are some possible reasons?
- Comment on any other aspects of your findings that are interesting to you.

```
In [48]: # Don't change this helper function!
def PlotAttributeShift(data2, pic_output):
    sample_index = random.sample(range(1, len(data2)), 16)

    fig, axs = plt.subplots(4, 8)
    fig.set_figheight(10)
    fig.set_figwidth(15)

    for i in range(4):
        for j in range(4):
            axs[i, 2*j].imshow(data2[sample_index[4*i+j], :, :, :])
            axs[i, 2*j].axis('off')
            axs[i, 2*j+1].imshow(np.clip(pic_output[sample_index[4*i+j]], 0, 1))
            axs[i, 2*j+1].axis('off')
```

```
In [49]: def AttributeShift(encoder_model, decoder_model, data1, data2):

    z_code_1 = encoder_model.predict(data1)
    z_code_2 = encoder_model.predict(data2)

    # Calculate means in latent space
    mean_z_1 = np.mean(z_code_1, axis=0)
    mean_z_2 = np.mean(z_code_2, axis=0)

    # Shift vector
```

```

shift_vector = mean_z_1 - mean_z_2

# Apply shift vector
shifted_z_codes = z_code_2 + shift_vector

# Output using decoder
pic_output = decoder_model.predict(shifted_z_codes)

print(f"Shift vector shape: {shift_vector.shape}")
print("---- Attribute Shift Complete ----")

return pic_output

```

```

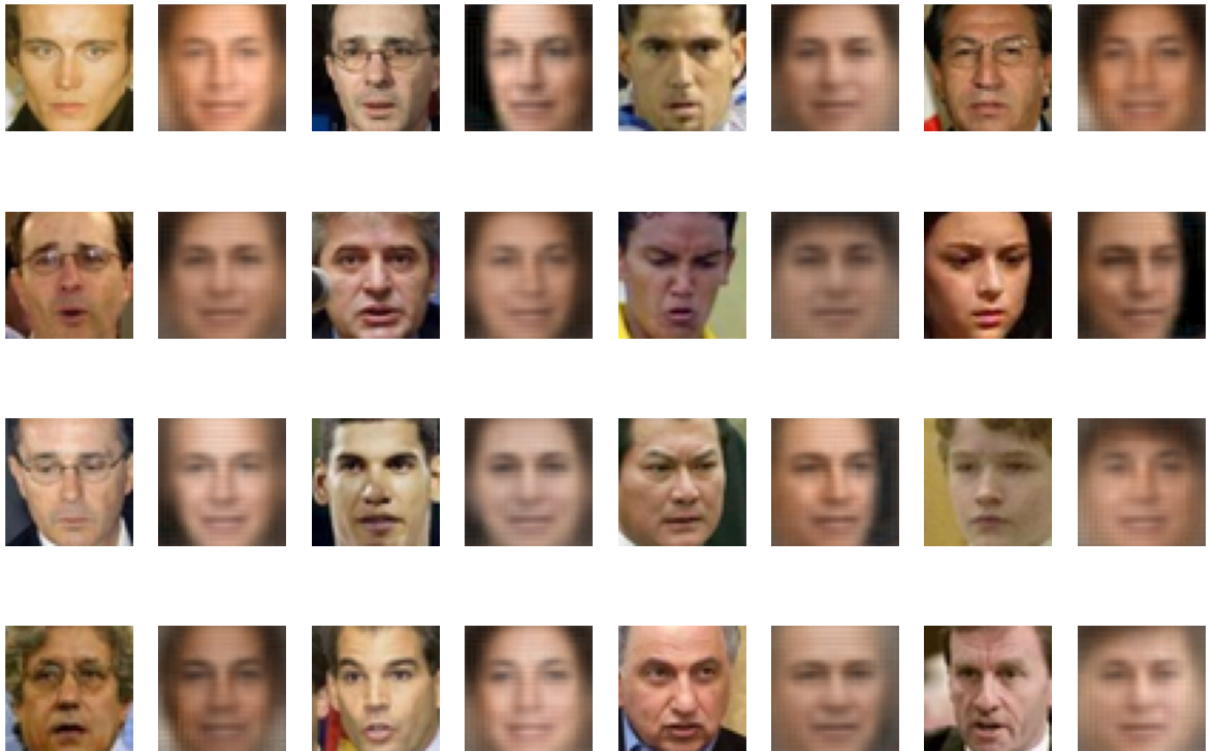
In [50]: pic_output = AttributeShift(encoder_model,decoder_model,smile_data,no_smile_
PlotAttributeShift(no_smile_data,pic_output)

```

```

2/2 ----- 0s 26ms/step
2/2 ----- 0s 25ms/step
2/2 ----- 0s 29ms/step
Shift vector shape: (100,)
---- Attribute Shift Complete ----

```



```

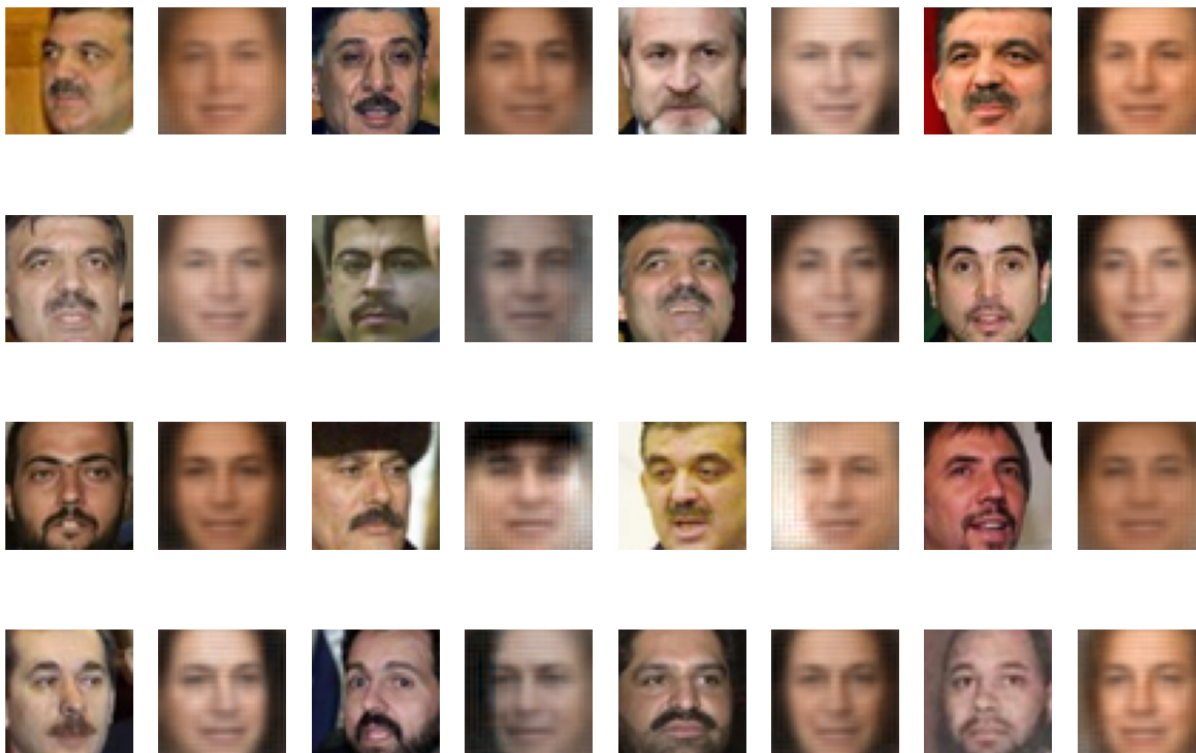
In [51]: pic_output = AttributeShift(encoder_model,decoder_model,smile_data,mustache_
PlotAttributeShift(mustache_data,pic_output)

```

```

2/2 ----- 0s 27ms/step
2/2 ----- 0s 27ms/step
2/2 ----- 0s 28ms/step
Shift vector shape: (100,)
---- Attribute Shift Complete ----

```



```
In [52]: pic_output = AttributeShift(encoder_model,decoder_model,male_data,female_data)
PlotAttributeShift(female_data,pic_output)
```

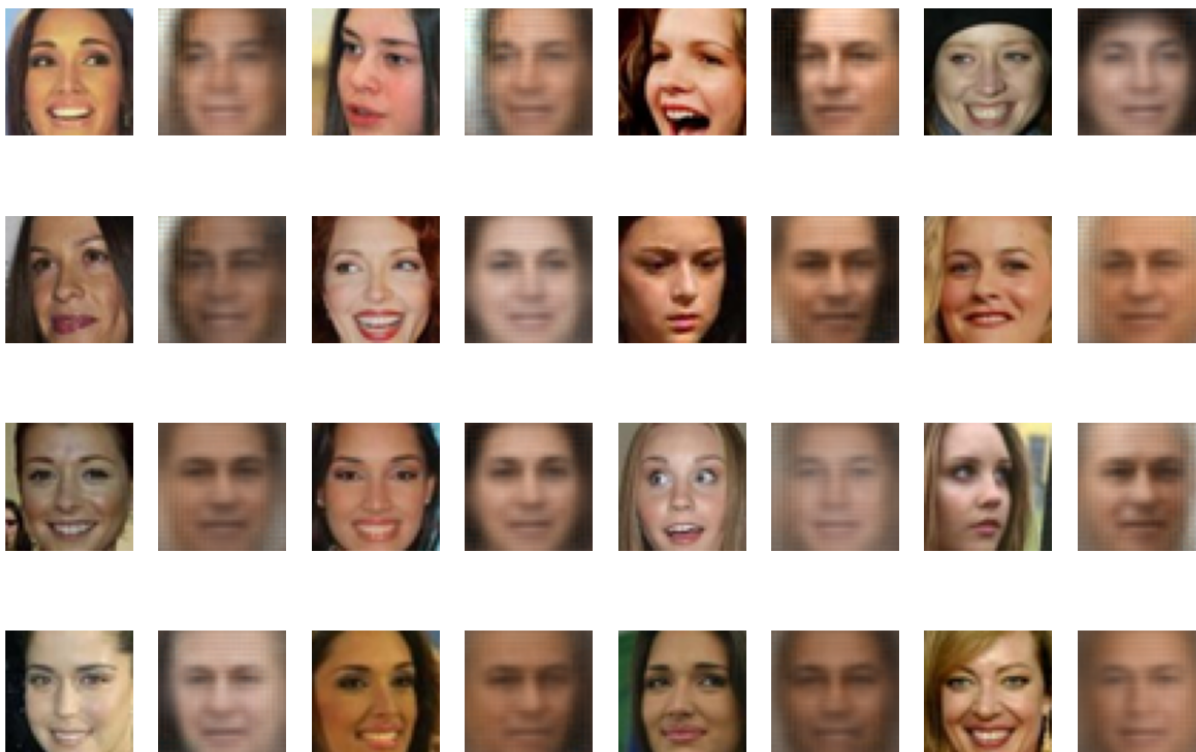
2/2 ————— 0s 28ms/step

2/2 ————— 0s 34ms/step

2/2 ————— 0s 25ms/step

Shift vector shape: (100,)

--- Attribute Shift Complete ---



## 1.5 Response

Do the generated faces look like what you expected? If not, can you think of some possible reasons? Do the faces with new attributes resemble the original faces? If not, can you think of some possible reasons? Which of the attribute shift is more successful? What are some possible reasons? Comment on any other aspects of your findings that are interesting to you.

Overall, the generated faces look about what I expected. In line with the previous results, we see that the generated faces are quite generic and more closely tied to the distribution that we're mixing with than the original picture. This can be seen most clearly in the final shift, which is from female to male: the generated faces look more male than female (to my eye). We see that, although all of the attribute shifts have this problem, the other two (no smile to smile and mustache to smile) do somewhat better. For example, I think the mustache to smile actually looks pretty good, maybe because there were enough faces in the smile set similar to those in the mustache set. One thing I find interesting is that the female to male shift actually maintains skin tone quite well during the shift.

## 1.6 Generating new faces (3 points)

Variational autoencoders can be used to generate new data; this is why they are generative models. We can sample new data points from the distribution in latent space and reconstruct new, fake faces based on them.

To draw a sample close to an existing sample in the latent space, we can add a scaled random sample from the normal distribution to the latent space code of an existing sample. The scalar, which we call the `noise_level` is a parameter that we can tune.

Run `GenerateFaces` with three different values of `noise_level` and comment on the generated faces. (3 points)

- Do the generated images look like faces?
- What happens when the new samples diverge more from the existing samples?  
What is a possible reason?

```
In [53]: def GenerateFaces(data, LATENT_SPACE_SIZE, noise_level):  
         sample_index = random.sample(range(1, len(data)), 15)  
         latent_space = noise_level*np.random.normal(size=(15, LATENT_SPACE_SIZE))  
         generated_image = decoder_model.predict(latent_space)  
         fig = plt.figure(figsize=(15,10))
```

```

for i in range(15):
    ax = fig.add_subplot(3, 5, i+1)
    ax.imshow(np.clip(generated_image[i, :, :, :], 0, 1))
    ax.axis('off')

```

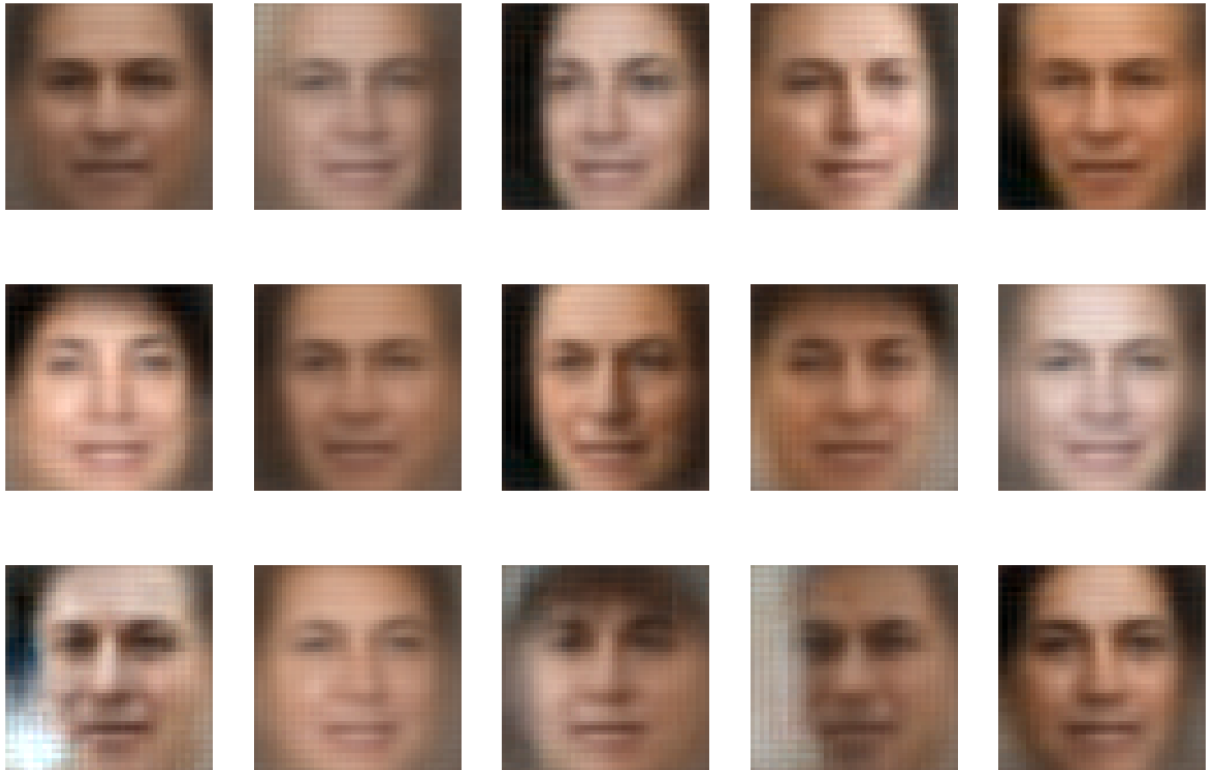
```

In [54]: LATENT_SPACE_SIZE = 100
GenerateFaces(data, LATENT_SPACE_SIZE, 0.1)

```

1/1 ————— 0s 75ms/step

1/1 ————— 0s 55ms/step



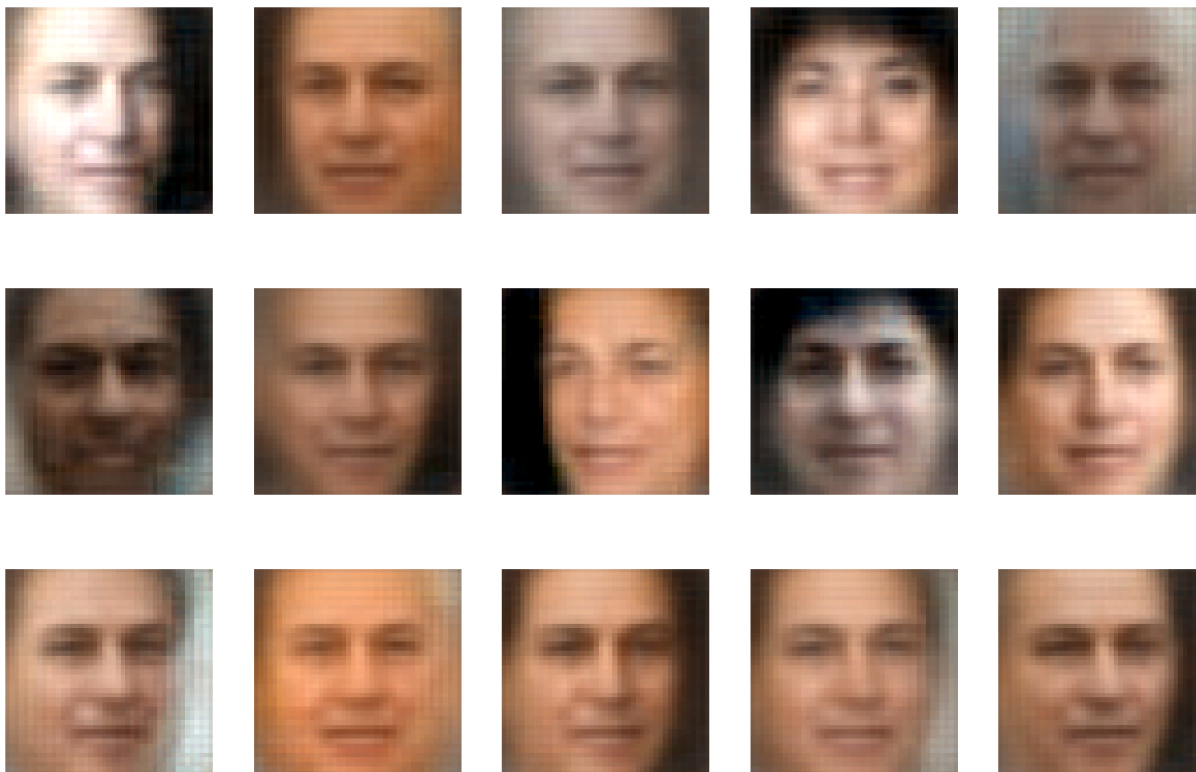
```

In [55]: GenerateFaces(data, LATENT_SPACE_SIZE, 0.5)

```

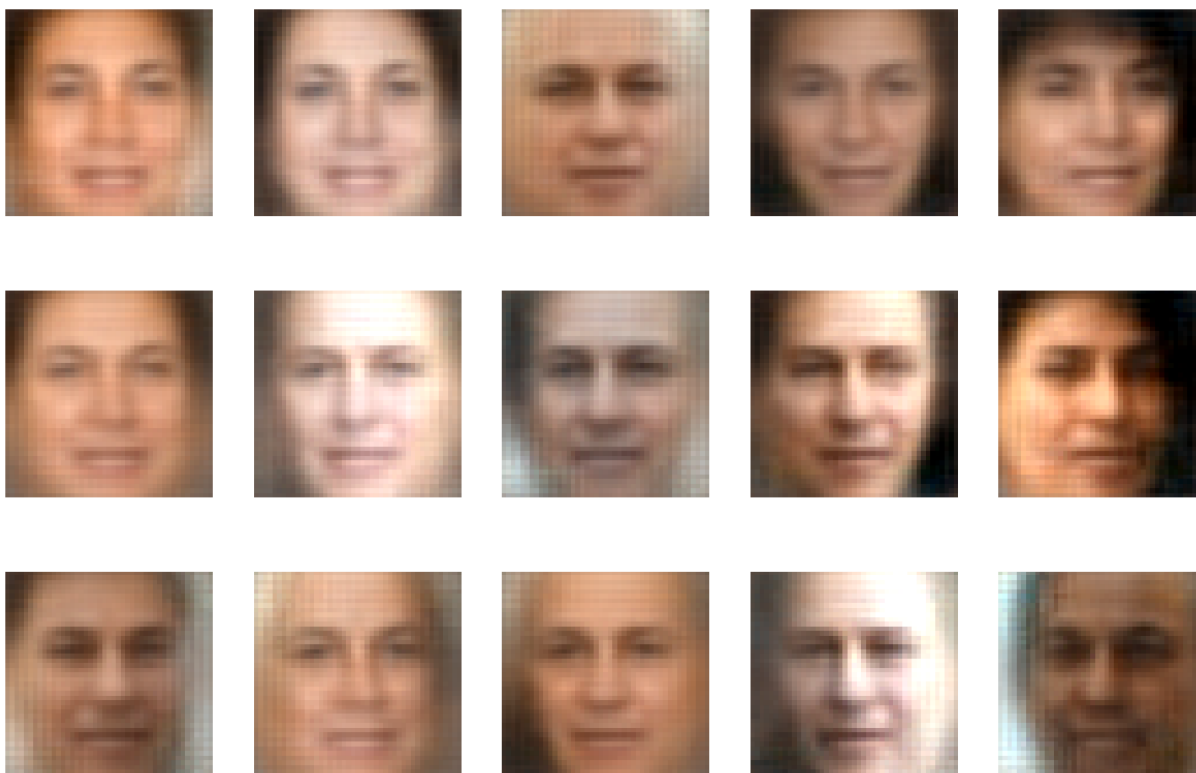
1/1 ————— 0s 62ms/step

1/1 ————— 0s 54ms/step



In [56]: `GenerateFaces(data,LATENT_SPACE_SIZE,1)`

1/1  0s 56ms/step  
 1/1  0s 53ms/step



1.6 Response

## Do the generated images look like faces? What happens when the new samples diverge more from the existing samples? What is a possible reason?

The generated images do look like faces, although there are few details. But, we at least see the outline of the mouth, nose, eyes, and overall face shape, as well as skin tone, for example. As the noise level increases and the new samples diverge more from the existing samples, we see that the images still look like faces, although the features begin changing quite a bit. This is because we are adding randomness at the latent space level. So we still get identifiable faces, they are just more random.

### Optional: Changing the loss function (2 points extra credit)

The variable `factor` is a parameter of the loss function. In this optional problem, you will play with it and think about its effect on the performance of VAE.

Retrain the model with a smaller factor. Repeat 1.2, 1.6 (using `noise_level = 1`) and latent space visualization. Comment on how the reconstruction results, generated new faces and latent space distributions change. (2 points)

- Which model reconstructs the faces better?
- Do the generated faces look different?
- How do the latent variable distributions differ?
- Do the differences make sense? Can you explain what you observed?

```
In [57]: # Your code here
```

[Your markdown here]

## Problem 2: Gaussian neighborhood regression (10 points)

In multivariate Gaussians, edges are about *direct* relationships after controlling for all other variables. The conditional law of a single coordinate  $X_i$  given the rest  $X_{-i}$  reveals which variables directly “enter” the model for  $X_i$ . Algebraically, this local behavior is encoded by the  $i$ th row/column of the precision matrix  $\Omega = \Sigma^{-1}$ . Graphical Lasso estimates a sparse precision matrix  $\hat{\Omega}$  by maximizing the Gaussian log-likelihood with an  $\ell_1$  penalty on off-diagonal entries. Zeros in  $\hat{\Omega}$  correspond to missing edges (conditional independences) in the learned graph.

Let  $X = (X_1, \dots, X_p)^\top \sim \mathcal{N}_p(0, \Sigma)$ , and write  $\Omega = \Sigma^{-1}$ . For  $i \in \{1, \dots, p\}$ , denote  $-i = \{1, \dots, p\} \setminus \{i\}$ .

## 2.1 Conditional law of one node.

Show that  $X_i \mid X_{-i}$  is Gaussian with a linear mean in  $X_{-i}$ . Express the regression coefficients  $\beta_{i \leftarrow -i} \in \mathbb{R}^{p-1}$  and the conditional variance  $\sigma_{i|-i}^2$  in terms of  $\Omega$ .

*Hint.* You may *complete the square* on  $x^\top \Omega x$ , or use a *Schur complement* derivation. After permuting coordinates so that  $i$  is first, partition

$$\Sigma = \begin{bmatrix} a & b^\top \\ b & C \end{bmatrix}, \quad a = \Sigma_{ii} \in \mathbb{R}, \quad b = \Sigma_{-i,i} \in \mathbb{R}^{p-1}, \quad C = \Sigma_{-i,-i} \in \mathbb{R}^{(p-1) \times (p-1)}.$$

**Schur complement.** If  $C$  is invertible (it is, since  $\Sigma \succ 0$ ), the Schur complement of  $C$  in  $\Sigma$  is

$$S := a - b^\top C^{-1} b.$$

**Block inversion formula.** Then

$$\Sigma^{-1} = \begin{bmatrix} S^{-1} & -S^{-1} b^\top C^{-1} \\ -C^{-1} b S^{-1} & C^{-1} + C^{-1} b S^{-1} b^\top C^{-1} \end{bmatrix}.$$

2.1

Let  $X = (X_1, \dots, X_p)' \sim N(0_p, \Sigma)$

Following the hint, partition  $\Sigma$  so that the  $i^{\text{th}}$  var. comes first:

$$\Sigma = \begin{bmatrix} a & b' \\ b & c \end{bmatrix}, \text{ where}$$

$$a = \Sigma_{ii}, b = \Sigma_{-i,i}, c = \Sigma_{-i,-i}$$

So that

$$\begin{bmatrix} X_i \\ X_{-i} \end{bmatrix} \sim N\left(\begin{bmatrix} 0 \\ 0_{p-1} \end{bmatrix}, \begin{bmatrix} a & b' \\ b & c \end{bmatrix}\right)$$

Now, the conditional distribution is

$$E[X_i | X_{-i}] = b' c^{-1} X_{-i}, \text{Var}(X_i | X_{-i}) = a - b' c^{-1} b$$

$$\text{Let } \beta_{i|-i} := c^{-1} b \text{ and } \sigma_{i|-i}^2 := a - b' c^{-1} b$$

$$\text{so we have } X_i | X_{-i} \sim N(\beta_{i|-i} X_{-i}, \sigma_{i|-i}^2)$$

Using the block inversion formula,

$$\Omega = \Sigma^{-1} = \begin{bmatrix} s^{-1} & -s^{-1} b' c^{-1} \\ -c^{-1} b s^{-1} & c^{-1} + c^{-1} b s^{-1} b' c^{-1} \end{bmatrix}$$

$$\text{where } S := a - b' c^{-1} b$$

Now we can rewrite the components of  $\Omega$ :

$$\Omega_{ii} = S^{-1} = (\sigma_{i|-i}^2)^{-1}, \Omega_{i,-i} = -S^{-1} b' c^{-1}$$

$$\Rightarrow b' c^{-1} = -\frac{\Omega_{i,-i}}{\Omega_{ii}}$$

$$\therefore E(X_i | X_{-i}) = b' C^{-1} X_{-i} = -\frac{\Omega_{i,-i}}{\Omega_{ii}} X_{-i}$$

$$\Rightarrow \sigma_{i|-i}^2 = \frac{1}{\Omega_{ii}}$$

$$\text{So, finally, } X_i | X_{-i} \sim N\left(-\frac{\Omega_{i,-i}}{\Omega_{ii}} X_{-i}, \frac{1}{\Omega_{ii}}\right)$$

$$\text{with } \beta_{i \leftarrow -i} = -\frac{\Omega_{i,-i}}{\Omega_{ii}}, \quad \sigma_{i|-i}^2 = \frac{1}{\Omega_{ii}}$$

2.2 | By 2.1, we have  $\beta_{i \leftarrow -i} = -\frac{\Omega_{i,-i}}{\Omega_{ii}}$ , so

$$\beta_{i \leftarrow j} = 0 \Leftrightarrow \frac{-\Omega_{i,j}}{\Omega_{ii}} = 0$$

$$\Leftrightarrow \Omega_{i,j} = 0$$

Now, partition indices as  $i, j, r := \{1, \dots, p\} \setminus \{i, j\}$

$$\text{WTS } \Omega_{i,j} = 0 \Leftrightarrow X_i \perp X_j | X_r$$

We know that conditional distribution of  $(X_i, X_j)$  is still normal:

$$(X_i, X_j) | X_r \sim N(\mu_{ij|r}, \Sigma_{ij,ij}^{-1}), \text{ where}$$

$$\Sigma_{ij,ij} = \begin{pmatrix} \Omega_{ii} & \Omega_{ij} \\ \Omega_{ji} & \Omega_{jj} \end{pmatrix}$$

$$\text{Then } \text{Cov}(X_i, X_j | X_r) = (\Sigma_{ij,ij}^{-1})_{12} = -\frac{\Omega_{ij}}{\Omega_{ii}\Omega_{jj} - \Omega_{ij}^2}$$

$$\therefore \text{Cov}(X_i, X_j | X_r) = 0 \Leftrightarrow \Omega_{ij} = 0$$

For bivariate Gaussians, zero conditional covariance is equivalent to conditional independence, so

$$X_i \perp X_j | X_r \Leftrightarrow \Omega_{ij} = 0$$

## 2.2 One neighbor at a time.

Fix  $j \neq i$ . First prove that the coefficient of  $X_j$  in the mean of  $X_i | X_{-i}$  equals zero if and only if  $\Omega_{ij} = 0$ . Then deduce (Gaussian case) that

$$\Omega_{ij} = 0 \iff X_i \perp X_j | X_{-(i,j)}.$$

*Hint:* Use the result from 2.1. "iff" ( $\Leftrightarrow$ ) means: prove **both** directions ( $\Rightarrow$  and  $\Leftarrow$ ).

$$\therefore E\{X_i | X_{-i}\} = b' C^{-1} X_{-i} = -\frac{\Omega_{i,-i}}{\Omega_{ii}} X_{-i}$$

$$\Rightarrow \sigma_{i|-i}^2 = \frac{1}{\Omega_{ii}}$$

$$\text{So, finally, } X_i | X_{-i} \sim N\left(-\frac{\Omega_{i,-i}}{\Omega_{ii}} X_{-i}, \frac{1}{\Omega_{ii}}\right)$$

$$\text{with } \beta_{i \leftarrow -i} = -\frac{\Omega_{i,-i}}{\Omega_{ii}}, \quad \sigma_{i|-i}^2 = \frac{1}{\Omega_{ii}}$$

2.2 | By 2.1, we have  $\beta_{i \leftarrow -i} = -\frac{\Omega_{i,-i}}{\Omega_{ii}}$ , so

$$\beta_{i \leftarrow j} = 0 \Leftrightarrow \frac{-\Omega_{i,j}}{\Omega_{ii}} = 0$$

$$\Leftrightarrow \Omega_{i,j} = 0$$

Now, partition indices as  $i, j, r := \{1, \dots, p\} \setminus \{i, j\}$

$$\text{WTS } \Omega_{i,j} = 0 \Leftrightarrow X_i \perp X_j | X_r$$

We know that conditional distribution of  $(X_i, X_j)$  is still normal:

$$(X_i, X_j) | X_r \sim N(\mu_{ij|r}, \Sigma_{ij,ij}^{-1}), \text{ where}$$

$$\Sigma_{ij,ij} = \begin{pmatrix} \Omega_{ii} & \Omega_{ij} \\ \Omega_{ji} & \Omega_{jj} \end{pmatrix}$$

$$\text{Then } \text{Cov}(X_i, X_j | X_r) = (\Sigma_{ij,ij}^{-1})_{12} = -\frac{\Omega_{ij}}{\Omega_{ii}\Omega_{jj} - \Omega_{ij}^2}$$

$$\therefore \text{Cov}(X_i, X_j | X_r) = 0 \Leftrightarrow \Omega_{ij} = 0$$

For bivariate Gaussians, zero conditional covariance is equivalent to conditional independence, so

$$X_i \perp X_j | X_r \Leftrightarrow \Omega_{ij} = 0$$

## 2.3 Factorization and graph structure

Suppose every coefficient in the mean of  $X_i | X_{-i}$  is zero (i.e.,  $\beta_{i \leftarrow -i} = 0$ ). Show that the joint density factorizes as  $p(x) = p(x_i) p(x_{-i})$ . Briefly interpret the implication **for the graph structure**.

2.3 From 2.1, we know that  $\beta_{i,-i} = 0 \Rightarrow \Omega_{i,-i} = 0$

$$\Rightarrow \Omega = \begin{bmatrix} \Omega_{:,i} & 0 \\ 0 & \Omega_{-i,-i} \end{bmatrix}$$

$$\begin{aligned} \text{Then } \log p(x) &= -\frac{1}{2} x' \Omega x + \text{constant} \\ &= -\frac{1}{2} (\Omega_{:,i} x_i^2 + x_{-i}' \Omega_{-i,-i} x_{-i}) + \text{constant} \end{aligned}$$

$$\begin{aligned} (\text{After normalizing...}) \Rightarrow p(x) &= \left[ (2\pi)^{-\frac{p}{2}} \Omega_{:,i}^{\frac{1}{2}} \exp\left(-\frac{1}{2} \Omega_{:,i} x_i^2\right) \right] \\ &\quad \cdot \left[ (2\pi)^{-(p-1)/2} |\Omega_{-i,-i}|^{\frac{1}{2}} \exp\left(-\frac{1}{2} x_{-i}' \Omega_{-i,-i} x_{-i}\right) \right] \\ &= p(x_{-i}) p(x_i) \end{aligned}$$

$$\text{So } p(x) = p(x_{-i}) p(x_i)$$

Graphically, zeros in  $\Omega$  correspond to missing edges.  
Here, all  $\Omega_{i,-i} = 0$ , so  $X_i$  is independent of all  $X_{-i}$ ; i.e., it has no edges connecting it to any other nodes.  $X_i \perp X_{-i}$

### Problem 3: Taking stock (15 points)

A joint distribution of data has a natural graph associated with it. When the distribution is multivariate normal, this graph is encoded in the pattern of zeros and non-zeros in the inverse of the covariance matrix, also known as the "precision matrix."

In class we demonstrated the graphical lasso for estimating the graph on ETF data. In

this problem you will construct two different "portfolios" of stocks, and run the graphical lasso to estimate a graph, commenting on your results.

All of the code you might need for this is contained in the demo.

## Data provided

As demonstrated in class, you will run on equities data downloaded from Yahoo Finance. Your job is to construct two "portfolios" of stocks, each with some organizing principle (e.g., sectors, regions, indices vs. equities). Each portfolio must contain **at least 20 stocks**. The page [https://en.wikipedia.org/wiki/List\\_of\\_S%26P\\_500\\_companies](https://en.wikipedia.org/wiki/List_of_S%26P_500_companies) has GICS sectors, which you may find useful for the glasso problem.

You are provided two CSV files containing stock prices **since 2021-09-10 (post-COVID start)**:

- [full\\_stock\\_daily.csv](#) (rows = dates, columns = **ticker** symbols; daily close)
- [full\\_stock\\_weekly.csv](#) (rows = dates, columns = **ticker** symbols; weekly close = last available trading day of the week)

**Note on NAs.** Some symbols begin trading after 2021-09-10, so their early entries will be NA . Handle missingness during preprocessing (see checklist below).

You can look up prices by ticker and date directly from these CSVs. If a needed symbol is **not** in the CSVs, use the provided **R file** `stock_price_from_yahoo_finance.r` to fetch it (via `BatchGetSymbols`) and merge it into your working data.

### Preprocessing checklist (concise):

- Select your symbols; **align dates** across symbols.
- For the analysis window you choose, **filter** symbols with poor coverage (e.g., keep  $\geq 90$ –95% non-missing).
- Build a **complete-cases** matrix for Graphical Lasso (no NAs across chosen symbols/dates).

```
In [58]: stock_dat = pd.read_csv("/content/drive/MyDrive/full_stock_daily.csv", index
print(stock_dat.head(3))
```

|            | A          | AA        | AACG | AAL       | AAME | AAMI  | AAOI | \ |
|------------|------------|-----------|------|-----------|------|-------|------|---|
| date       |            |           |      |           |      |       |      |   |
| 2021-09-10 | 177.229996 | 49.500000 | 2.85 | 18.950001 | 3.92 | 26.65 | 7.43 |   |
| 2021-09-13 | 172.080002 | 49.220001 | 2.87 | 19.309999 | 3.90 | 27.00 | 7.45 |   |
| 2021-09-14 | 172.419998 | 48.000000 | 2.83 | 19.209999 | 3.91 | 26.60 | 7.21 |   |

|            | AAON      | AAP        | AAPL       | ... | BRZE | CRGO | IREN | LUNR |
|------------|-----------|------------|------------|-----|------|------|------|------|
| \          |           |            |            |     |      |      |      |      |
| date       |           |            |            | ... |      |      |      |      |
| 2021-09-10 | 43.380001 | 200.029999 | 148.970001 | ... | NaN  | NaN  | NaN  | NaN  |
| 2021-09-13 | 43.346668 | 199.259995 | 149.550003 | ... | NaN  | NaN  | NaN  | NaN  |
| 2021-09-14 | 43.613335 | 201.169998 | 148.119995 | ... | NaN  | NaN  | NaN  | NaN  |

|            | SSM | ADAMZ | ARBKL | SG  | FINW | LITM |
|------------|-----|-------|-------|-----|------|------|
| date       |     |       |       |     |      |      |
| 2021-09-10 | NaN | NaN   | NaN   | NaN | NaN  | NaN  |
| 2021-09-13 | NaN | NaN   | NaN   | NaN | NaN  | NaN  |
| 2021-09-14 | NaN | NaN   | NaN   | NaN | NaN  | NaN  |

[3 rows x 5328 columns]

```
In [59]: portfolio_1 = ['MO', 'CPB', 'CLX', 'KO', 'CL',
                        'STZ', 'COST', 'DG', 'DLTR', 'EL',
                        'GIS', 'HSY', 'HRL', 'KDP', 'KHC',
                        'KR', 'LW', 'MKC', 'TAP', 'PEP']

portfolio_2 = ['AES', 'LNT', 'AEP', 'AWK', 'ATO',
               'CNP', 'CMS', 'ED', 'XEL', 'D',
               'DTE', 'DUK', 'EIX', 'ETR', 'EVRG',
               'ES', 'EXC', 'FE', 'NRG', 'SO']
```

```
In [60]: def make_portfolio_df(prices: pd.DataFrame, tickers: list,
                               start_date: str = "2021-09-10", min_coverage: float =
                               .....
                               Keep dates >= 2021-09-10 and only tickers with >=90% non-missing coverage
                               Returns a complete-cases matrix (no NAs).
                               .....
    df = prices.copy()
    if not isinstance(df.index, pd.DatetimeIndex):
        df.index = pd.to_datetime(df.index, errors="coerce")
    df = df.sort_index()

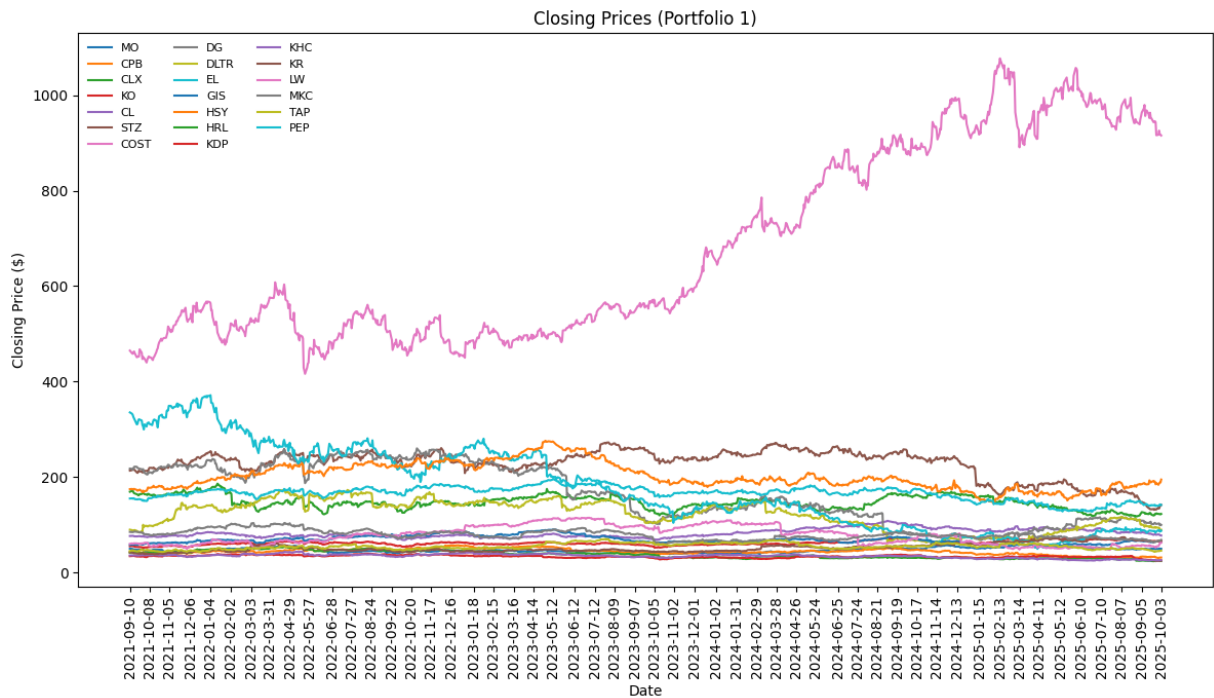
    # restrict dates & tickers
    tickers = [t for t in tickers if t in df.columns]
    df = df.loc[pd.to_datetime(start_date):, tickers]

    # coverage check
    cov = df.notna().mean()
    good = cov[cov >= min_coverage].index.tolist()
    dropped = sorted(set(tickers) - set(good))
    if dropped:
        print(f"[Info] Dropping low-coverage (<{min_coverage:.0%}) tickers:")

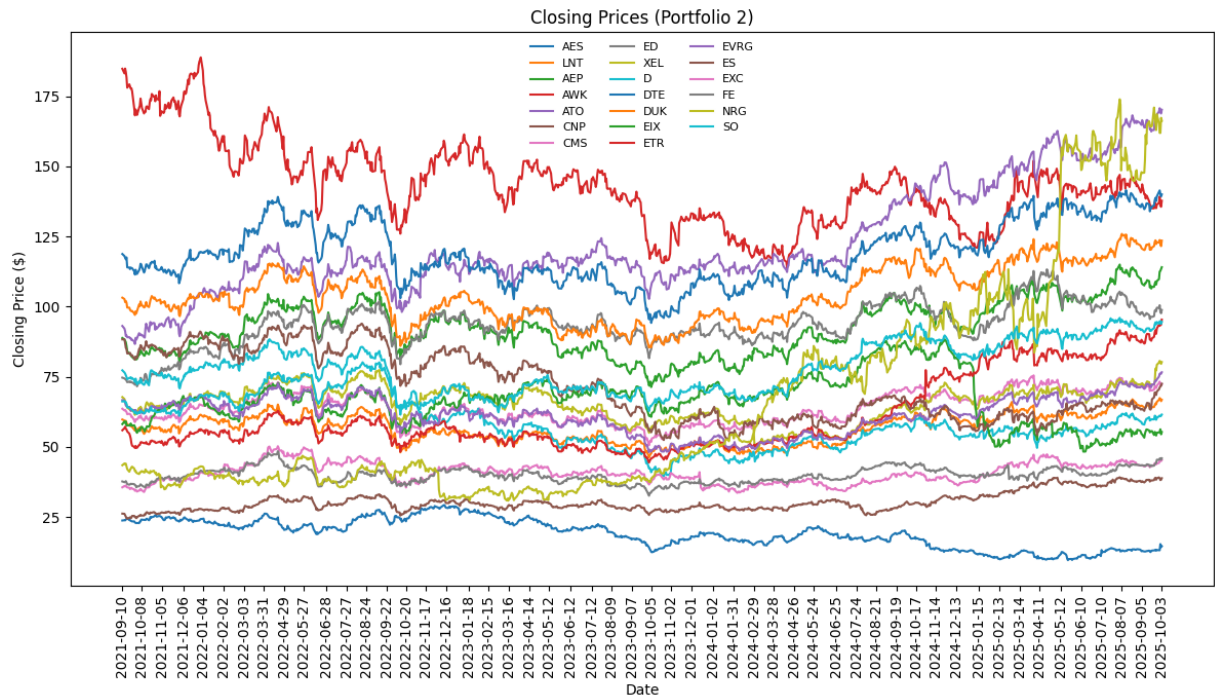
    # return complete cases
    return df[good].dropna()
```

```
# --- 2) build price datasets
p1_prices = make_portfolio_df(stock_dat, portfolio_1)
p2_prices = make_portfolio_df(stock_dat, portfolio_2)
```

```
In [61]: plt.figure(figsize=(12, 7))
plt.plot(p1_prices.index, p1_prices)
plt.xlabel('Date')
plt.ylabel('Closing Price ($)')
plt.title('Closing Prices (Portfolio 1)')
plt.xticks(p1_prices.index[::20], rotation=90)
plt.legend(p1_prices.columns, fontsize=8, ncol=3, frameon=False)
plt.tight_layout()
plt.show()
```



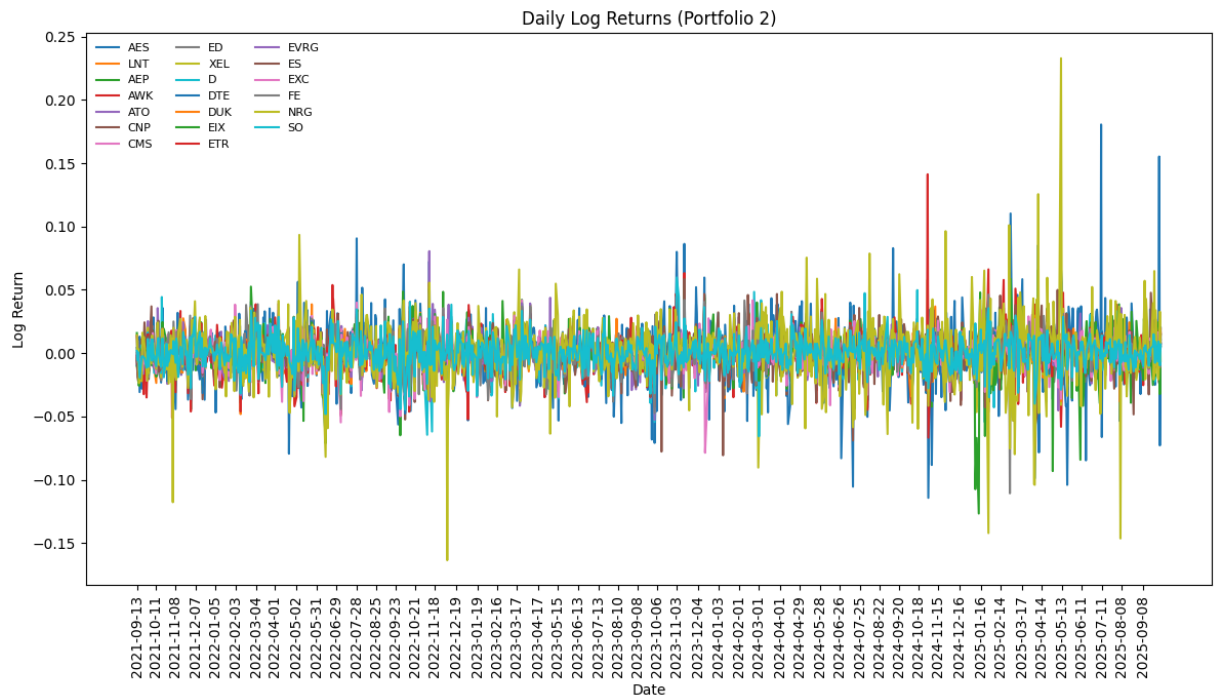
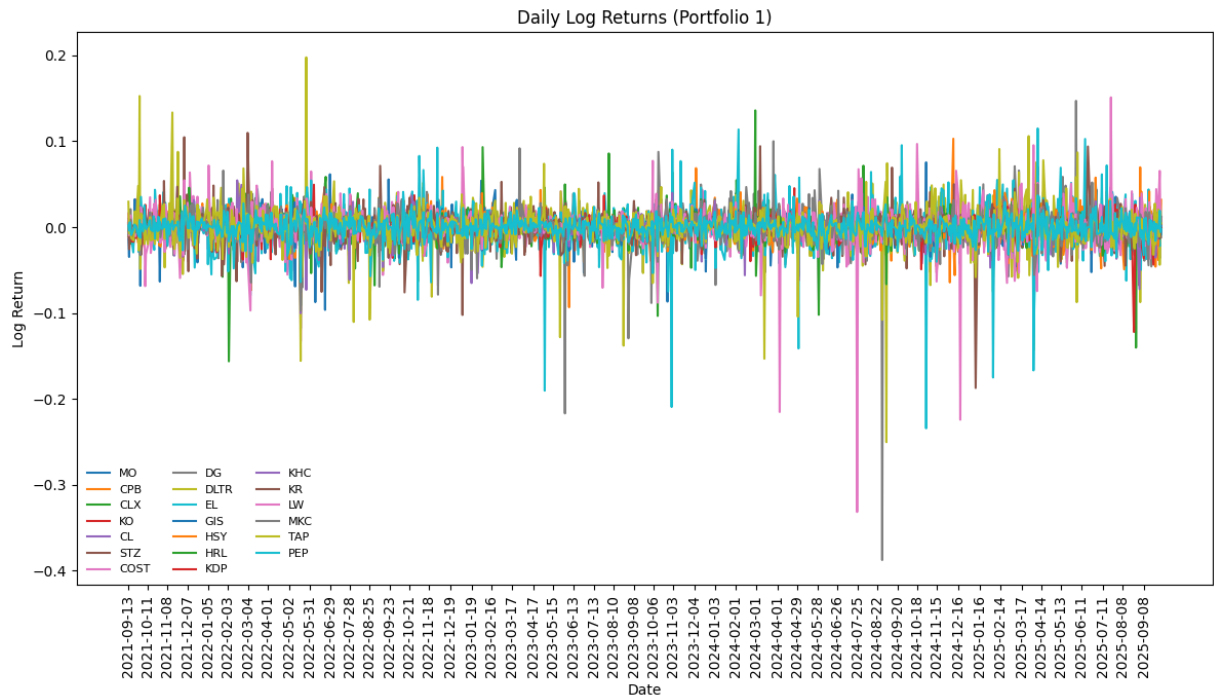
```
In [62]: plt.figure(figsize=(12, 7))
plt.plot(p2_prices.index, p2_prices)
plt.xlabel('Date')
plt.ylabel('Closing Price ($)')
plt.title('Closing Prices (Portfolio 2)')
plt.xticks(p2_prices.index[::20], rotation=90)
plt.legend(p2_prices.columns, fontsize=8, ncol=3, frameon=False)
plt.tight_layout()
plt.show()
```



```
In [63]: # --- Compute log returns for each portfolio
p1_logreturn = np.log1p(p1_prices.pct_change()).iloc[1:]
p2_logreturn = np.log1p(p2_prices.pct_change()).iloc[1:]
```

```
In [64]: # --- Plot Portfolio 1 log returns
plt.figure(figsize=(12, 7))
plt.plot(p1_logreturn.index, p1_logreturn)
plt.xlabel("Date")
plt.ylabel("Log Return")
plt.title("Daily Log Returns (Portfolio 1)")
plt.xticks(p1_logreturn.index[::20], rotation=90)
plt.legend(p1_logreturn.columns, fontsize=8, ncol=3, frameon=False)
plt.tight_layout()
plt.show()

# --- Plot Portfolio 2 log returns
plt.figure(figsize=(12, 7))
plt.plot(p2_logreturn.index, p2_logreturn)
plt.xlabel("Date")
plt.ylabel("Log Return")
plt.title("Daily Log Returns (Portfolio 2)")
plt.xticks(p2_logreturn.index[::20], rotation=90)
plt.legend(p2_logreturn.columns, fontsize=8, ncol=3, frameon=False)
plt.tight_layout()
plt.show()
```

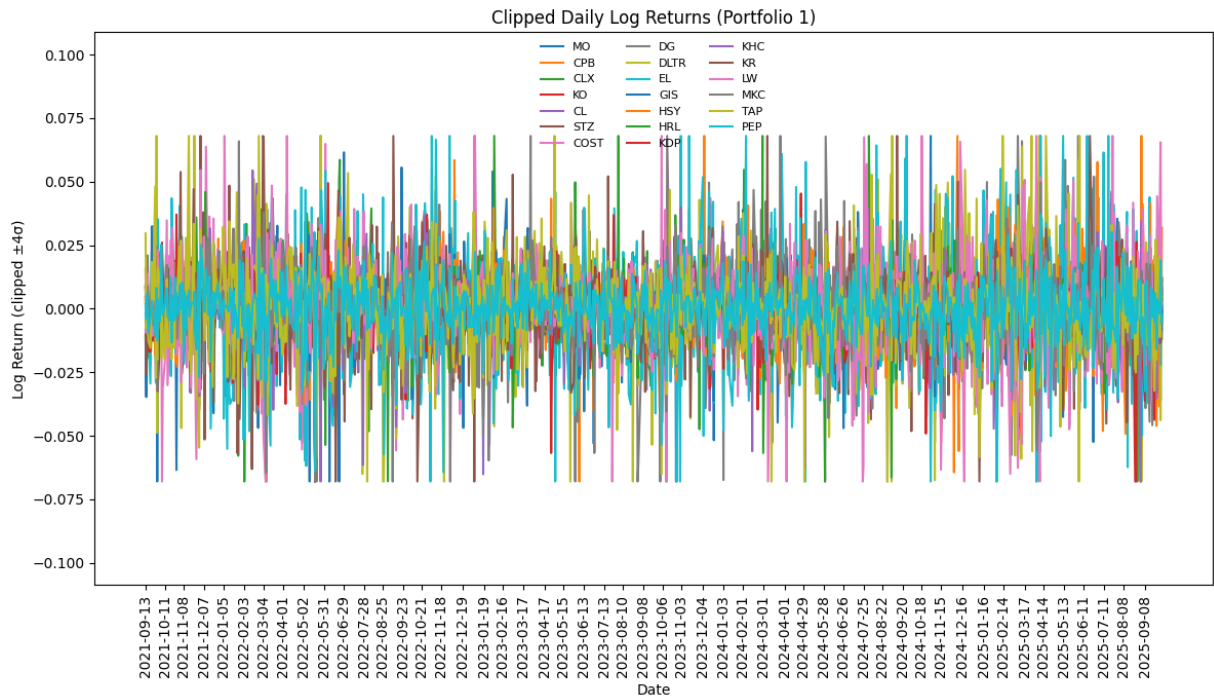


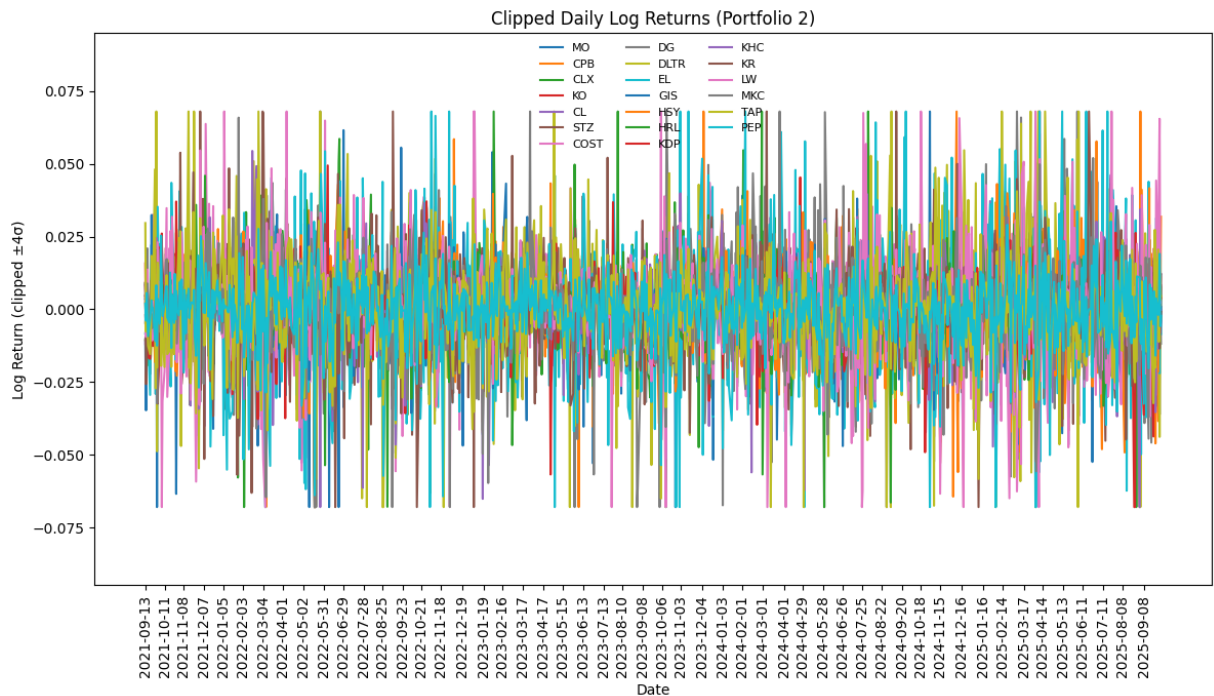
```
In [65]: def clip_portfolio_logreturns(prices_df, sigma_mult=4):
# 1) log returns
logret = np.log1p(prices_df.pct_change()).iloc[1:]
# 2) global mean & std across all stocks and dates (same limit for all)
g_std = logret.values.std(ddof=1)
lower, upper = -1 * sigma_mult*g_std, sigma_mult*g_std
# 3) clip
clipped_vals = np.clip(logret.values, lower, upper)
return logret, logret.__class__(clipped_vals, index=logret.index, column

# --- Portfolio 1
p1_logreturn, p1_clipped = clip_portfolio_logreturns(p1_prices, sigma_mult=4
ymin, ymax = p1_clipped.min().min(), p1_clipped.max().max())
```

```
padding = 0.3 * (ymax - ymin) # 30% padding
plt.figure(figsize=(12, 7))
plt.plot(p1_clipped.index, p1_clipped)
plt.xlabel("Date")
plt.ylabel("Log Return (clipped  $\pm 4\sigma$ )")
plt.title("Clipped Daily Log Returns (Portfolio 1)")
plt.xticks(p1_clipped.index[::20], rotation=90)
plt.legend(p1_clipped.columns, fontsize=8, ncol=3, frameon=False)
plt.ylim(ymin - padding, ymax + padding)
plt.tight_layout()
plt.show()

# --- Portfolio 2
p2_logreturn, p2_clipped = clip_portfolio_logreturns(p2_prices, sigma_mult=4
ymin, ymax = p2_clipped.min().min(), p2_clipped.max().max()
padding = 0.3 * (ymax - ymin) # 30% padding
plt.figure(figsize=(12, 7))
plt.plot(p1_clipped.index, p1_clipped)
plt.xlabel("Date")
plt.ylabel("Log Return (clipped  $\pm 4\sigma$ )")
plt.title("Clipped Daily Log Returns (Portfolio 2)")
plt.xticks(p1_clipped.index[::20], rotation=90)
plt.legend(p1_clipped.columns, fontsize=8, ncol=3, frameon=False)
plt.ylim(ymin - padding, ymax + padding)
plt.tight_layout()
plt.show()
```





```
In [66]: def run_glasso_and_plot(logret_df: pd.DataFrame, alpha=0.75, title=""):
        """
        Enforce complete cases (no NA rows), drop zero-variance columns,
        standardize features, fit Graphical Lasso, and plot the precision heatmap
        """

        # Copy & sort by date
        R = logret_df.copy()
        if not isinstance(R.index, pd.DatetimeIndex):
            R.index = pd.to_datetime(R.index, errors="coerce")
        R = R.sort_index()

        # --- Complete cases across chosen symbols/dates
        n0 = len(R)
        p0 = R.shape[1]
        R = R.dropna(how="any")          # drop any rows (dates) with missing values
        dropped_rows = n0 - len(R)

        # --- Drop zero-variance columns (after NA removal)
        stds = R.std(axis=0, ddof=1)
        zero_var = stds[stds == 0].index.tolist()
        if zero_var:
            R = R.drop(columns=zero_var)

        # Final sanity
        if R.isna().any().any():
            raise ValueError("Data still contains NaNs after dropna(). Check early on!")
        if R.shape[1] == 0:
            raise ValueError("All columns were dropped (zero variance).")

        if dropped_rows or zero_var:
            print(f"[Info] dropped {dropped_rows} rows for complete cases; "
                  f"dropped zero-variance cols: {zero_var}")

        # --- Standardize features (across time)
```

```

X = (R - R.mean(0)) / R.std(0, ddof=1)
X = X.to_numpy()

# --- Fit Graphical Lasso
glasso = GraphicalLasso(alpha=alpha, max_iter=1000, tol=1e-4).fit(X)

# --- Precision with zeroed diagonal for visualization
Omega = glasso.precision_.copy()
np.fill_diagonal(Omega, 0.0)

# --- Plot heatmap with ticker labels that survived filtering
tickers = R.columns.tolist()
Omega_df = pd.DataFrame(Omega, index=tickers, columns=tickers)

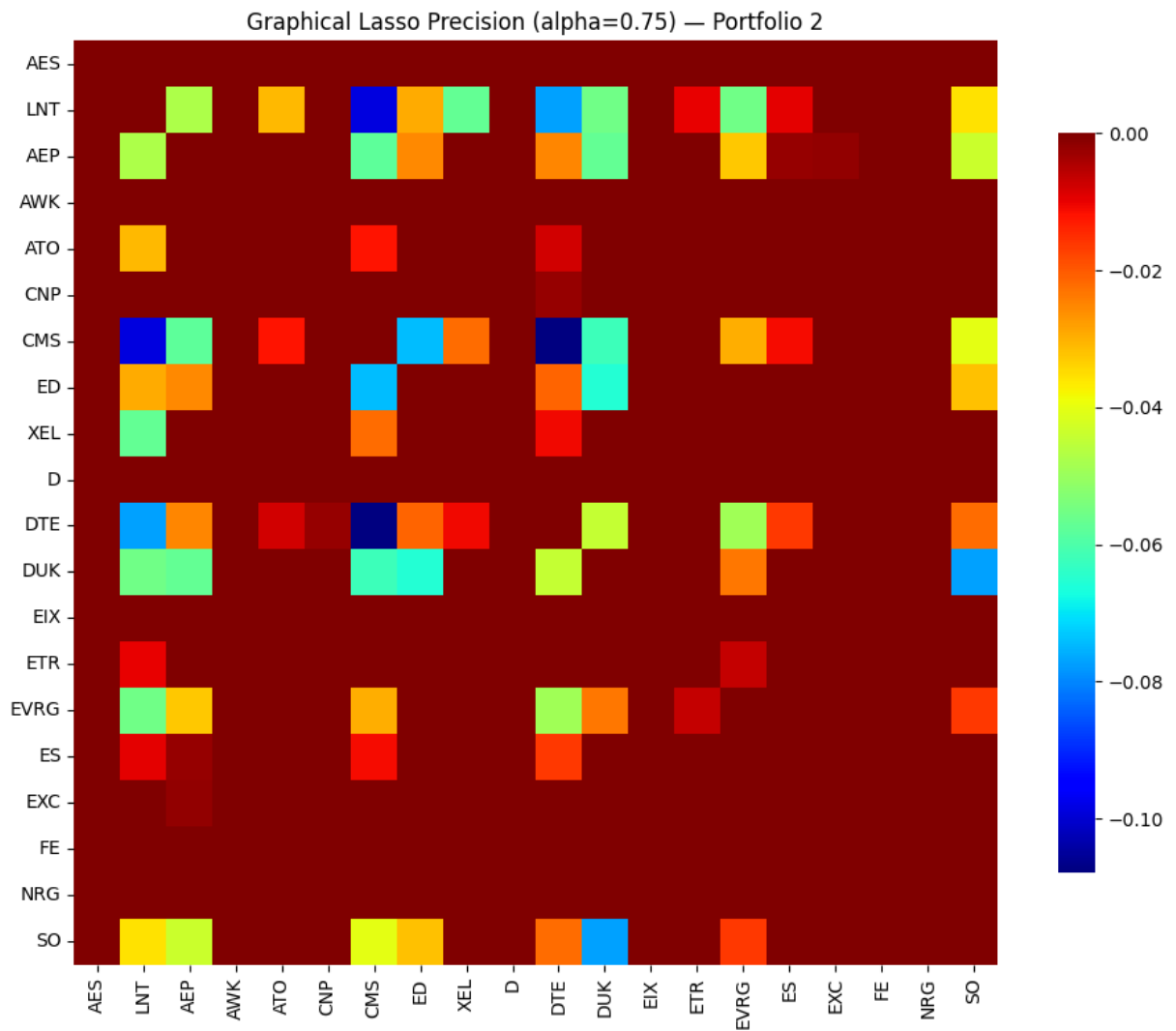
plt.figure(figsize=(10, 8))
sns.heatmap(Omega_df, cmap="jet", square=True, cbar_kws={"shrink": 0.8})
plt.title(f"Graphical Lasso Precision (alpha={alpha}) - {title}")
plt.tight_layout()
plt.show()

return glasso, Omega_df

# --- Run on both portfolios (using the globally clipped log returns created
gl1, Omega1 = run_glasso_and_plot(p1_clipped, alpha=0.5, title="Portfolio 1")
gl2, Omega2 = run_glasso_and_plot(p2_clipped, alpha=0.75, title="Portfolio 2")

```





```
In [67]: import numpy as np
import pandas as pd
import networkx as nx
import matplotlib.pyplot as plt

def build_glasso_graph(Omega_df: pd.DataFrame, thresh: float = 0.02, title:
    """
    Build and plot a network from a precision matrix.
    - Omega_df: DataFrame (tickers x tickers), typically with diagonal zeroes
    - thresh: keep edges with |precision| > thresh
    """
    # Long-form edge list
    links = (
        Omega_df.stack()
        .rename("value")
        .reset_index()
        .rename(columns={"level_0": "var1", "level_1": "var2"})
    )
    # Drop self-loops & apply threshold on absolute strength
    links = links[(links["var1"] != links["var2"]) & (links["value"].abs() >
    thresh)]

    # Build graph with attributes
    G = nx.from_pandas_edgelist(
```

```

        links, source="var1", target="var2",
        edge_attr=["value"], create_using=nx.Graph()
    )

    # Layout: use abs(weight) as spring weights (stronger edges pull more)
    # Add a deterministic seed for reproducibility
    np.random.seed(42)
    weight_map = { (u, v): abs(d["value"]) for u, v, d in G.edges(data=True) }
    pos = nx.spring_layout(G, k=1.7/np.sqrt(max(len(G.nodes()),1)),
                          iterations=200, seed=42, weight=None) # start un
    # one pass of adjustment based on weights (optional fine-tune)
    for _ in range(50):
        pos = nx.spring_layout(G, pos=pos, weight=None, iterations=1, seed=4

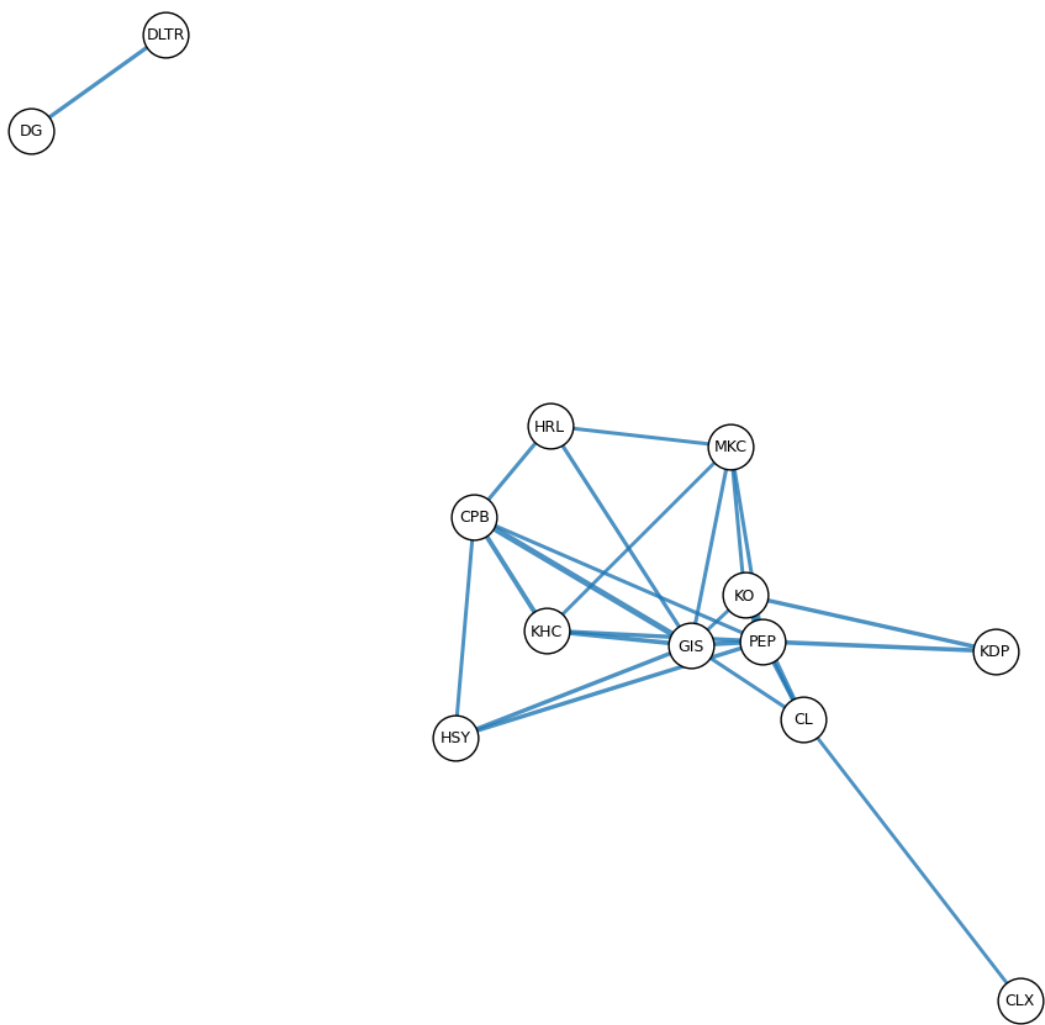
    # Edge styling
    edges = G.edges(data=True)
    widths = [2.0 + 6.0*abs(d["value"]) for _, _, d in edges] # magnitud
    colors = ["tab:red" if d["value"] > 0 else "tab:blue" for _, _, d in edg

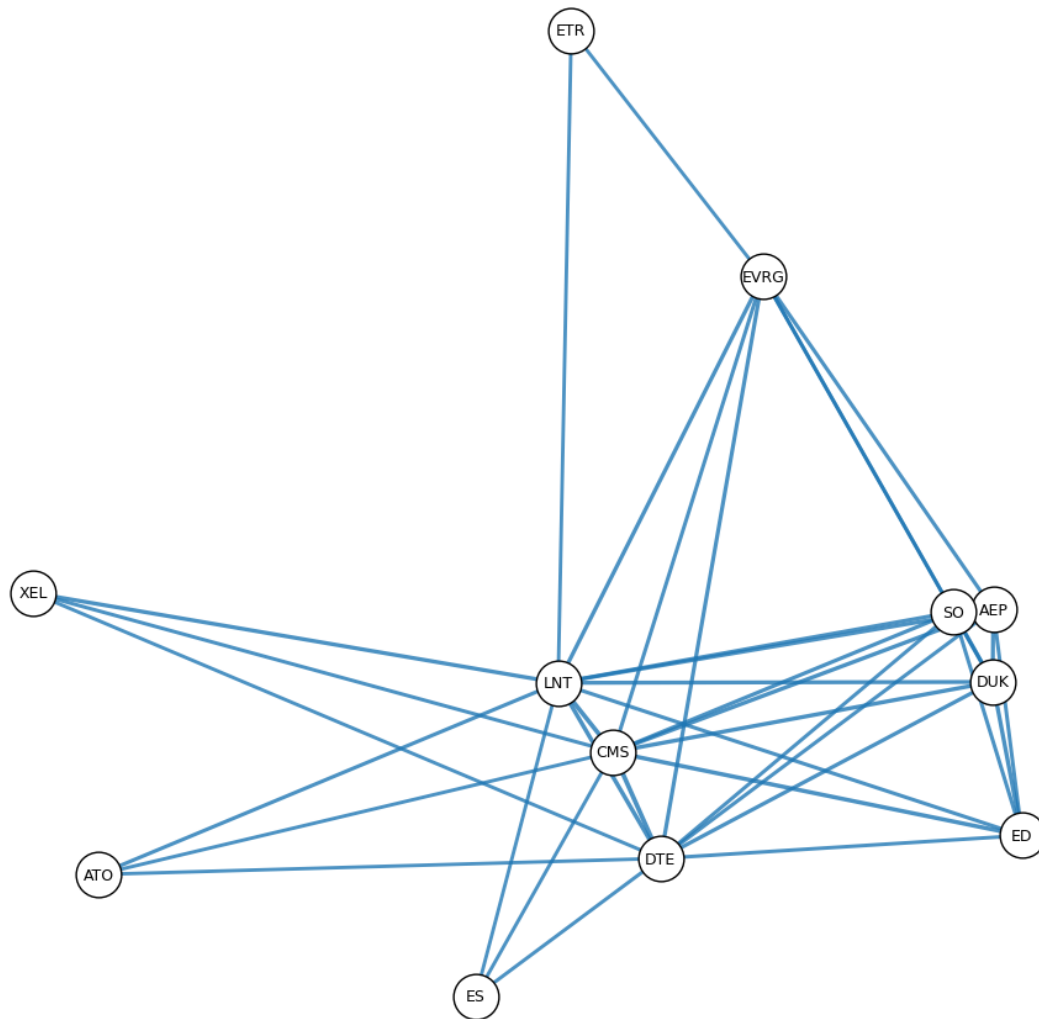
    # Draw
    plt.figure(figsize=(10, 10))
    nx.draw_networkx_nodes(G, pos, node_size=700, node_color="white", edgeco
    nx.draw_networkx_labels(G, pos, font_size=9)
    nx.draw_networkx_edges(G, pos, width=widths, edge_color=colors, alpha=0.
    plt.title(f"Glasso network ( $|\Omega| > \{\text{thresh}\}\) - \{title\}")
    plt.axis("off")
    plt.tight_layout()
    plt.show()

    return G, links

# --- Use with your precision matrices (from run_glasso_and_plot) ---
# Omega1 and Omega2 are DataFrames of the precision matrices with tickers as
G1, E1 = build_glasso_graph(Omega1, thresh=0.005, title="Portfolio 1")
G2, E2 = build_glasso_graph(Omega2, thresh=0.005, title="Portfolio 2")$ 
```

Glasso network ( $|\Omega| > 0.005$ ) — Portfolio 1





## Analyzing your portfolios

Your task is to analyze each portfolio using the graphical lasso, and comment on your findings. Here are the types of questions you should address:

- How did you choose the portfolio? How did you choose the date range and frequency (daily, weekly, etc.)? Remember, each of the portfolios must contain at least 20 stocks, and be organized in some reasonable way.
- Display the graph obtained with the graphical lasso, using networkx. How did you choose the regularization level? Does the structure of the graph make sense? Is it sensitive to the choice of regularization level? Is this the structure you expected to see when you designed the portfolio? Why or why not?
- What are some of the conditional independence assumptions implied by the graph?

Are some parts of the graph more densely connected than others? Why?

## Portfolio 1 - Consumer Staples

**How did you choose the portfolio? How did you choose the date range and frequency (daily, weekly, etc.)?**

For my first portfolio, I handpicked a sample of 20 stocks in the "consumer staples" industry. These include some well-known companies like Coca-Cola (KO) and Pepsi (PEP), as well as some others I had not heard of before (e.g., Hormel Foods (HRL)). I thought this industry might be interesting since, despite the Pepsi & Coke examples above, a lot of the companies seem to be quite disparate in both the types of products they make and the potential customers they might serve. On the other hand, maybe these companies' performances are more interrelated than I might think.

I chose the daily frequency along with a date range past 2021 so that I wasn't missing any entries. All else equal, I figured daily over monthly was better since there would be more information for the glasso to use.

**How did you choose the regularization level? Does the structure of the graph make sense? Is it sensitive to the choice of regularization level? Is this the structure you expected to see when you designed the portfolio? Why or why not?**

This portfolio needed a much lower regularization level than portfolio 2 (see below), which I think speaks to my guess that the stock performance of these companies may not be highly correlated. When creating the graph, I chose an extremely low cutoff for an edge, just so I could see as many connections as possible. The overall structure seems not to depend too heavily on this level, which is reassuring. The first thing that jumps out to me is that Pepsi and Coke are close together -- a good sign. They seem to be at the center of a cluster that includes companies like General Mills (GIS). We see that household products are also connected (e.g., Clorox through Colgate-Palmolive). Finally, Dollar General (DG) and Dollar Tree (DLTR) are connected in their own grouping, which aligns with my expectations.

**What are some of the conditional independence assumptions implied by the graph? Are some parts of the graph more densely connected than others?**

## Why?

So, for instance, one conditional independence implication that might make sense is that Clorox is independent of the entire grouping of General Mills et al., conditional on the return for Colgate. That grouping of GIS, PEP, and KO is the densest part of the graph. Another conditional independence implication is that Keurig-Dr.Pepper (KDP) enters the cluster only through Pepsi and Coke. Overall, the graph follows our intuition quite well :).

## Portfolio 2 - Utilities Companies

### How did you choose the portfolio? How did you choose the date range and frequency (daily, weekly, etc.)?

For my second portfolio, I handpicked a sample of 20 stocks in the "utilites" industry. These include less companies that I was already familiar with, but I did recognize a few (e.g., NRG Energy [NRG], Southern [SO], American Electrical Power [AEP]). I picked this industry because I didn't really have a prior about whether these companies' stock performances should be very correlated or not. On the one hand, it's easy to think of events that effect large parts of the U.S. utilities industry at the same time (e.g., heat waves, tornado/hurricane seasons, etc.). On the other hand, regulation, policies, and infrastructure vary tremendously across different parts of the U.S., so maybe the companies' performances would not be very connected.

Again, I chose the daily frequency along with a date range past 2021 so that I wasn't missing any entries. As above, I figured daily over monthly should be better.

### How did you choose the regularization level? Does the structure of the graph make sense? Is it sensitive to the choice of regularization level? Is this the structure you expected to see when you designed the portfolio? Why or why not?

For this portfolio, I kept a relatively high regularization level of 0.75, which still revealed quite a few nonzero covariances. In that sense, the industry's stock performances do seem quite correlated. Fortunately, it seems that the overall graph structure is not very sensitive to the regularization level (just the number of edges). The structure here is a bit tougher to analyze thoughtfully since I know less about the utilities industry. It seems like the graph does not quite map neatly onto geographic regions of the U.S. (at least using the headquarters as a proxy for the region a given company might serve).

**What are some of the conditional independence assumptions implied by the graph? Are some parts of the graph more densely connected than others? Why?**

Overall, we see all of the companies that appear in the graph (i.e., have at least one nonzero covariance) have at least two edges. So we get now conditional independence results where one company can be predicted given only one other company in the graph. One conditional independence implication is that Entergy (ETR) is separated from the main cluster via Evergy (EVRG) and Alliant Energy (LNT). All three companies are electric companies, but their headquarters are in different regions of the U.S., so it's hard to say whether this result makes sense. Overall, the densest part of the graph is the cluster of SO, AEP, and DUK and the cluster of LNT, CMS, and DTE. Again, it's not clear to me that these represent geographic regions, so it's very tough to say why these are the densest clusters.